ONLINE SHOPPING SYSTEM

A mini project report submitted in partial fulfilment of the requirement for the award of the degree of

BACHELOR OF TECHNOLOGY

in

COMPUTER SCIENCE AND ENGINEERING (AI&ML)

Submitted by

SANJANA (5241412020)

Under the Esteemed Guidance of

Ch. Bhanu Kumar
Assistant Professor



DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING (AI&ML)

ENGINEERING AND TECHNOLOGY PROGRAM

GAYATRI VIDYA PARISHAD COLLEGE FOR DEGREE AND PG COURSES (A)

Rushikonda, Visakhapatnam – 45 (Approved by AICTE | Accredited by NBA | Accredited by NAAC | Affiliated to Andhra University) An ISO 9001:2015 Certified Institution | 2026-2027 |

DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING (AI&ML)

GAYATRI VIDYA PARISHAD COLLEGE FOR DEGREE AND PG COURSES (A)

Rushikonda, Visakhapatnam - 45

CERTIFICATE

This is to certify that the project report entitled **"ONLINE SHOPPING SYSTEM"** being submitted by **SANJANA (5241412020)** in the partial fulfilment for the award of the Degree of Bachelor of Technology in Computer Science and Engineering to the Gayatri Vidya Parishad College for Degree and PG Courses (A), Visakhapatnam is a record of bonafide work carried out under my guidance and supervision.

Project Guide	**Head of the Department**
Ch. Bhanu Kumar Assistant Professor	**Dr. N.V. Ramana Murthy** Professor M. Tech, Ph. D

External Examiner

VISION AND MISSION OF THE INSTITUTE

VISION

Creating Human Excellence for a Better Society

MISSION

Unfold into a World class organization with strong academic and research base, producing responsible citizens to cater to the changing needs of the society.

VISION AND MISSION OF THE DEPARTMENT

VISION

“Create and sustain as a Centre of excellence in Computer Science and Engineering and allied areas of research to pave the way for providing better technocrats to the society.”

MISSION

“To provide strong conceptual base (M1) for acquiring programming skills, to develop high computational innovative skills (M2) and to empower leadership qualities with strong ethical base (M3)”

PROGRAM EDUCATIONAL OBJECTIVES (PEOs)

PEO1: Ability to develop hardware and software problem solving capabilities by inculcating strong fundamental, analytical, and communication skills.

PEO2: Ability to build confidence to address the challenges in their work and to apply innovative technical tools through interaction with professional bodies.

PEO3: Ability to gain sufficient depth of knowledge, Self-learning skills, technical skills and leadership qualities through participating in personality development activities, so as to sustain in multidisciplinary environment.

PEO4: To be able to respond to societal needs in professional and ethical concerns.

PROGRAM SPECIFIC OUTCOMES (PSOs)

PSO 1: System Inception and Elaboration: Conceptualize the software and/or hardware systems, system components and processes/procedures through requirement analysis, Modelling/Design of the system using various architectural/design patterns, Standard Notations, procedures and algorithms.

PSO 2: System Construction: Implement the systems, Procedures and Processes using the state-of-the-art technologies, standards, tools and Programming Paradigms.

PSO 3: System Testing and Deployment: Verify and Validate the Systems, Procedures and Processes using various testing and verification techniques and tools.

PSO 4: Quality and Maintenance: Manage the quality through various product development strategies under revision, transition and operation through maintainability, flexibility, testability, portability, reusability, interoperability, correctness, reliability, efficiency, integrity and usability to adapt the system to the changing structure and behaviour of the systems/environments.

DECLARATION

I hereby declare that the project entitled **"ONLINE SHOPPING SYSTEM"** submitted in partial fulfilment of the requirements for the award of Bachelor of Technology in Computer Science and Engineering, to Engineering and Technology Program, Gayatri Vidya Parishad College for Degree & PG Courses (A). I assure that this project is not submitted in any other University or College.

Name & Signature of the Student:
SANJANA (5241412020)

ACKNOWLEDGEMENT

With great pleasure I want to take this opportunity to express my heartfelt gratitude to all the people who helped in making this mini project work a grand success.

First of all I express my deep sense of gratitude to **Ch. Bhanu Kumar**, Assistant Professor, for his/her constant guidance throughout my project work.

I would like to thank **Dr. N.V. Ramana Murthy**, Professor, Head of the Department of Computer Science and Engineering (AI&ML), for being moral support throughout the period of my study.

I would like to thank all the Teaching and Non-Teaching staff of CSE (AI&ML) Department for sharing their knowledge with me.

SANJANA
(5241412020)

ABSTRACT

Online Shopping System is a web-based application designed to make buying and selling of goods over the internet simple, organised and transparent. Visiting shops in person costs time and travel, limits the choice of products and often ends in billing mistakes and manual record keeping. The proposed system replaces this with a digital shop where products, carts, orders and payments are handled in one place.

The system provides role-based access for two kinds of users: customers and administrators. A customer can register, log in, browse and search products, add them to a cart, check out with a shipping address, pay through a payment gateway (card or UPI), receive an order confirmation and view or cancel orders. The administrator can add, edit and remove products, update stock and prices, change the status of orders from Confirmed to Delivered, and block or unblock users.

The system was analysed and designed with UML using use case, class, activity, sequence, component and deployment diagrams, and was implemented as a working prototype named ShopLite using HTML, CSS and JavaScript. Stock is checked before every order, a declined payment never creates an order, and cancelling an order restores stock. The design is modular and can be extended with a Node.js/Java server, a MySQL database and a real payment gateway as shown in the deployment diagram.

TABLE OF CONTENTS

CERTIFICATE **2**

VISION AND MISSION OF THE INSTITUTE **3**

VISION AND MISSION OF THE DEPARTMENT **3**

PROGRAM EDUCATIONAL OBJECTIVES (PEOs) **4**

PROGRAM SPECIFIC OUTCOMES (PSOs) **4**

DECLARATION **5**

ACKNOWLEDGEMENT **6**

ABSTRACT **7**

TABLE OF CONTENTS **8**

1. INTRODUCTION **11**

1.1 Introduction to ONLINE SHOPPING SYSTEM: 11

1.2 Importance of Online Shopping 11

1.3 Objectives of the System 11

1.3.1 Scope of the Project 12

1.3.2 Benefits to Users and Administrators 12

1.3.3 Summary of Introduction 12

2. LITERATURE SURVEY **13**

2.1 Existing Shopping Systems: 13

2.2 Technologies Used in Similar Systems: 13

2.3 Gaps in Existing Systems: 13

2.4 Comparison of Shopping Systems (Features vs Limitations) 14

3. REQUIREMENTS AND ANALYSIS **15**

3.1 Existing system 15

3.2 Proposed system 15

3.3 Functional Requirements 16

3.3.1 Functional Requirements List 16

3.3.2 Use cases 16

3.4 Non-Functional Requirements: . . . 17
3.4.1 Software & Hardware Specifications . . . 18
3.5 ANALYSIS . . . 18
3.5.1 Functional Analysis . . . 18
3.5.2 Use Case Analysis . . . 18
3.5.3 Data Modelling . . . 19
3.5.4 Advantages of the Analysis Phase . . . 19
3.5.5 Limitations Identified . . . 19

4. SYSTEM DESIGN . . . 20
4.1 Design Goals . . . 20
4.2 System Architecture . . . 20
4.3 UML Diagrams . . . 21
4.4 Use Case Diagram . . . 21
4.5 Class Diagram . . . 22
4.6 Activity Diagram . . . 23
4.7 Sequence Diagram . . . 24
4.8 Component Diagram . . . 26
4.9 Deployment Diagram . . . 26
4.10 Database Design . . . 27

5. IMPLEMENTATION . . . 28
5.1 Technology & Coding Approach . . . 28
5.2 Database & Information Handling . . . 28
5.3 Programming Style . . . 28
5.4 Verification and Validation . . . 29
5.5 Implementation of Platform . . . 29
5.5.1 Order Placement and Stock Algorithm . . . 30
5.5.2 Security Implementation . . . 30
5.5.3 Error Handling . . . 30

6. TESTING . . . 32
6.1 Testing Activities : . . . 32
6.2 Testing Types : . . . 32
6.3 Test Cases: . . . 32

7. RESULTS & DEMONSTRATION . . . 34

7.1 User Registration and Login 34
7.2 Browsing Products 35
7.3 Cart 35
7.4 Checkout, Payment and Order 36
7.5 Admin Screens 37

8. SAMPLE CODE 40
8.1 Frontend & Implementation 40
8.2 Application Logic 41

9. CONCLUSION 43
9.1 Future Scope 43

10. REFERENCES 44

1. INTRODUCTION

1.1 Introduction to ONLINE SHOPPING SYSTEM:

Online Shopping System is a web-based platform designed to simplify and digitise the complete buying process between customers and the shop administrators. The platform enables customers to register themselves, browse and search products, add products to a cart, enter a shipping address, pay online and track or cancel their orders. Administrators, on the other hand, use a separate dashboard to add, edit and remove products, update stock, process orders and manage users.

In everyday life people buy clothes, electronics, books and household goods. Travelling to several shops to compare prices wastes time and fuel, shops close at fixed hours, and the choice is limited to what is on the shelf. Shop owners, in turn, still depend on paper bills, manual stock registers and cash, which are slow and prone to mistakes. By centralising products, carts, orders and payments in one system, the Online Shopping System removes most of this manual work.

The system was planned using the Unified Modelling Language (UML). Use case, class, activity, sequence, component and deployment diagrams were drawn in IBM Rational Software Architect Designer before coding started, and the design was then implemented as a working web application, ShopLite, using HTML, CSS and JavaScript.

1.2 Importance of Online Shopping

Traditional shopping lacks convenience, comparison and reliable records. Customers cannot see all products at once, shopkeepers allocate stock by guesswork, and the owner has no accurate information about how many items were sold or how much money was collected. Online shopping bridges this gap by giving every person the same live information at the same time.

E-commerce is one of the most important applications of information technology. An online shopping system saves the customer's time, gives a wider choice, avoids disputes over bills and payments, and gives the owner reliable data for planning purchases. For a small business, a website of this kind opens a market that is available all day, every day.

- Saves time and effort: customers browse and buy from anywhere without visiting a shop.
- Wider choice: all products with price and availability are shown in one place.
- Removes human error: totals, stock and order records are calculated by the system.
- Creates reliable records: every order and payment is stored permanently.
- Improves control: administrators can update prices and stock and see all orders instantly.

1.3 Objectives of the System

- Provide a centralised platform for product, cart, order and payment management.
- Allow customers to search products, add them to a cart and order without visiting a shop.
- Calculate the cart total automatically and check stock before the order is placed.
- Record payments (card or UPI), confirm the order and reduce stock after successful payment.
- Allow administrators to add, edit and remove products and to process orders and users.
- Protect the system with login and role-based access for customer and admin.
- Apply UML modelling during analysis and design and implement the design as a working system.

1.3.1 Scope of the Project

The Online Shopping System covers user registration and login, product browsing and search, cart management, checkout with shipping address, payment recording, order confirmation, order cancellation, order status updates and admin management of products and users. It supports two roles, the Customer and the Admin, and is suitable for small shops, college stores and startups.

The present version is a front-end prototype. It does not include a real payment gateway, e-mail or SMS notifications, product reviews, coupons or a server-side database; the data is stored in the browser and the payment gateway is simulated. These items are listed as future scope in Chapter 9.

1.3.2 Benefits to Users and Administrators

- Customers: live product and stock information, a simple cart, transparent totals and a history of all their orders.
- Administrators: full control over products, stock, prices, orders and users, and no paper records.
- Shop owners: better stock control, fewer disputes and reliable accounts.

1.3.3 Summary of Introduction

Online Shopping System aims to transform buying and selling from a manual, error-prone activity into a simple digital service by providing a user-friendly, secure and organised platform that connects customers and shop administrators.

2. LITERATURE SURVEY

2.1 Existing Shopping Systems:

Shopping systems can be grouped by how much of the work is done on the internet and how much in physical shops. The most common types are the traditional retail store, the large multi-vendor marketplace platform and the small single-seller website or social-media shop. Each of them solves part of the problem but also has limitations that motivated this project.

Traditional retail store. The customer visits the shop, selects goods from shelves and pays at the counter. The set-up is familiar and goods can be touched before buying, but the shop is open only at fixed hours, the choice is limited by shelf space, queues form at busy times, and billing and stock registers are maintained by hand.

Large multi-vendor marketplace platforms. These platforms list products from thousands of sellers, provide search, recommendations, online payment and delivery tracking. They offer a very wide choice, but they are complex, charge commissions from sellers, and a small shop has little control over how its products are shown or how its customers are served.

Single-seller websites and social-media shops. Small sellers use ready-made website builders or chat and social-media pages to take orders. The initial cost is low, but orders are often handled through messages, stock is not updated automatically, payments are collected manually and there is no proper order history or admin dashboard.

2.2 Technologies Used in Similar Systems:

- Front end: HTML, CSS and JavaScript (and frameworks such as React or Angular) for product pages, cart and checkout.
- Back end: Node.js with Express, Java with Spring Boot, or PHP/Python frameworks exposing REST APIs.
- Database: MySQL, PostgreSQL or MongoDB to store users, products, orders and payments.
- Payment: payment gateways supporting cards, net banking and UPI, called over secure HTTPS connections.
- Security: password hashing, session or token-based authentication and role-based access control.
- Hosting: cloud or local servers with HTTPS, and browser storage or caching for fast loading.

These technologies work together to give a fast, secure shopping experience. In this project the software layer, which stores products, carts, orders and payments and enforces the shopping rules, is implemented as a prototype; the production back end (server and MySQL database) is kept as future scope.

2.3 Gaps in Existing Systems:

- Traditional shops have limited hours, limited choice and manual records.
- Large marketplaces are complex and give small sellers little control.
- Social-media and chat-based selling has no automatic stock update, order history or admin dashboard.
- Many low-cost solutions do not enforce rules such as checking stock before an order or restoring stock on cancellation.

- Few simple systems combine browsing, cart, payment, order management and user management in a single place.

2.4 Comparison of Shopping Systems (Features vs Limitations)

System	Features	Limitations
Traditional retail store	Physical inspection of goods, immediate purchase, familiar process	Fixed hours, limited choice, queues, manual billing and stock registers
Large marketplace platform	Huge choice, search and recommendations, online payment, delivery tracking	Complex, commission charged to sellers, little control for small shops
Social-media / chat selling	Very low cost, quick to start, direct contact with the customer	Manual orders and payments, no automatic stock, no order history
Online Shopping System (proposed)	Web-based, product search, cart, checkout, payment, order tracking, admin dashboard, role-based login	Prototype stores data in the browser; payment gateway is simulated

From the comparison it is clear that a simple, low-cost, software-based system can fill the gap between complex marketplace platforms and informal manual selling. The Online Shopping System follows this idea.

3. REQUIREMENTS AND ANALYSIS

3.1 Existing system

Currently, shopping and shop management in most small shops is largely manual or fragmented:

- **Manual billing:** items are noted and totalled by hand, which is time-consuming and prone to errors.
- **Paper records and cash:** sales are written in registers and payments are collected in cash; bills are lost and amounts are miscalculated.
- **No live availability:** customers cannot know whether an item is in stock before they reach the shop.
- **Limited reporting:** owners cannot easily find sales, stock or revenue figures for a day or month.
- **Generic tools:** spreadsheets and general software are not designed for shopping and do not enforce rules such as stock checking.

3.2 Proposed system

The Online Shopping System provides a centralised digital environment to manage shopping efficiently:

- **Role-Based Screens:** separate screens for customers and administrators after secure login.
- **Product Catalogue:** products are shown with category, price and stock; customers can search and filter them.
- **Cart Management:** items can be added, increased, decreased and removed; the total is calculated automatically and stock limits are enforced.
- **Checkout and Payment:** the customer enters a shipping address and pays by card or UPI; a declined payment does not create an order.
- **Order Management:** confirmed orders reduce stock, can be cancelled by the customer (stock restored) and are moved through Packed, Shipped and Delivered by the admin.
- **Admin Control:** add, edit and remove products, update stock and prices, view all orders and block or unblock users.

3.3 Functional Requirements

3.3.1 Functional Requirements List

ID	Requirement	Description
FR1	User registration	A new user can register with name, email and a password of at least four characters.
FR2	Login / logout	A registered customer or admin can log in with email and password and can log out.
FR3	Browse products	The system shows all products with category, price and stock.
FR4	Search and filter	A customer can search by name and filter by category.
FR5	Manage cart	A customer can add a product to the cart, change the quantity (within stock) and remove it.
FR6	Checkout	A logged-in customer can check out the cart and enter a shipping address.
FR7	Make payment	A customer can pay by card or UPI; a declined payment creates no order.
FR8	Confirm order	After successful payment the system creates the order, records the payment and reduces stock.
FR9	Order history / cancel	A customer can view own orders and cancel a confirmed order; stock is restored.
FR10	Manage products	The admin can add, edit price and stock, and remove products.
FR11	Manage orders	The admin can view all orders and change the status (Confirmed, Packed, Shipped, Delivered, Cancelled).
FR12	Manage users	The admin can view users and block or unblock them; a blocked user cannot log in.

3.3.2 Use cases

Use cases describe the behaviour of the system from an actor's perspective:

- **Use Case Name: User Registration**
- Participating Actors: Customer
- Flow of Events: Customer provides name, email and password → System validates → Customer receives confirmation and is logged in
- Exit Condition: Customer is successfully registered
- Quality Requirements: Secure, fast and accurate registration; duplicate emails are rejected

- **Use Case Name: Login / Logout**
- Participating Actors: Customer, Admin
- Flow of Events: Actor enters email and password → System verifies → Screen for the actor's role is displayed
- Exit Condition: Actor is logged in (or logged out and returned to the shop)
- Quality Requirements: Wrong credentials and blocked accounts are rejected

- **Use Case Name: Browse and Search Product**
- Participating Actors: Customer
- Flow of Events: Customer opens the shop → types a search or selects a category → System lists matching products with price and stock
- Exit Condition: Matching products are displayed
- Quality Requirements: Fast response; out-of-stock products cannot be added

- **Use Case Name: Manage Cart**

- Participating Actors: Customer
- Flow of Events: Customer clicks Add to cart → System checks stock → Item is added; customer can change quantity or remove items
- Exit Condition: Cart shows items and total
- Quality Requirements: Quantity never exceeds stock; total is always correct

- **Use Case Name: Place Order and Make Payment**
- Participating Actors: Customer, Payment Gateway
- Flow of Events: Customer checks out → enters address and payment details → Gateway processes → System creates the order and reduces stock
- Exit Condition: Order is confirmed; if payment is declined no order is created
- Quality Requirements: An order is never created without successful payment

- **Use Case Name: Cancel Order**
- Participating Actors: Customer
- Flow of Events: Customer selects Cancel on a confirmed order → System changes status to Cancelled → stock is restored
- Exit Condition: Order status is Cancelled
- Quality Requirements: Only confirmed orders can be cancelled by the customer

- **Use Case Name: Manage Product**
- Participating Actors: Admin
- Flow of Events: Admin adds, edits or removes a product → System validates the values → Catalogue is updated
- Exit Condition: Updated product list is shown
- Quality Requirements: Invalid price or stock values are rejected

- **Use Case Name: Manage Order**
- Participating Actors: Admin
- Flow of Events: Admin selects a new status for an order → System updates it (stock is restored on cancellation)
- Exit Condition: Order shows the new status
- Quality Requirements: Status list is fixed to valid values

- **Use Case Name: Manage User**
- Participating Actors: Admin
- Flow of Events: Admin views users → blocks or unblocks a customer
- Exit Condition: User list is updated
- Quality Requirements: Admin accounts cannot be blocked

3.4 Non-Functional Requirements:

Non-functional requirements define the quality and performance of the system:

- **Dependability:** reliable performance without data loss; every change is saved to the data store immediately.

- **Quick Response:** fast loading of screens and immediate update of cart, stock and orders after every action.
- **User Friendliness:** an easy-to-use interface for both roles with clear messages for every error.
- **Scalability:** support for more products, users and orders.
- **Security:** role-based access control, input escaping and validation of every input; password hashing is planned for the server version.
- **Portability:** the system runs on any device with a modern browser; light and dark display modes are supported.
- **Maintainability:** modular code (separate HTML, CSS and JavaScript files) that can be extended with a server, database or payment gateway.

3.4.1 Software & Hardware Specifications

Item	Specification
Frontend	HTML5, CSS3, JavaScript (ES6)
Data storage	Browser localStorage (JSON); replaceable by MySQL
Modelling tool	IBM Rational Software Architect Designer
Code editor	Visual Studio Code with Live Server extension
Operating System	Windows 10 / 11; any browser supported (Chrome, Edge, Firefox)

Hardware Specifications (For Development/Server Setup)

- Development Machine: Laptop/Desktop with a modern browser and VS Code (4 GB RAM or more).
- Server (future): Cloud or local server for hosting the platform with a database.

3.5 ANALYSIS

The analysis phase defines the system requirements independently of the technology used for implementation. It focuses on modelling the system from the user's perspective and ensures the system meets its objectives.

3.5.1 Functional Analysis

The system is divided into multiple functional modules:

1. **User Management Module:** registration, login, logout and role-based access; blocked accounts are rejected.
2. **Product Catalogue Module:** product list, search and category filter; admin add, edit and remove.
3. **Cart Module:** add, change quantity, remove and total calculation with stock limits.
4. **Order Module:** order creation after payment, stock reduction, cancellation with stock restore and status updates.
5. **Payment Module:** accepts card or UPI details, simulates the gateway, records the payment with the order.
6. **Admin Dashboard Module:** product, order and user management screens.

3.5.2 Use Case Analysis

- Actors: Customer, Admin (and the Payment Gateway as an external system)
- Customer use cases: register, login, browse product, search, manage cart, place order, make payment, cancel order, logout.
- Admin use cases: login, manage product, manage order, manage user, logout.

Each use case includes: **Preconditions** (for example the customer must be logged in to check out and the cart must not be empty); **Trigger** (a user action such as clicking Add to cart or Pay now); and **Postconditions** (the action completes, the data is stored and the screen is updated).

3.5.3 Data Modelling

Entity Objects:

- User (ID, Name, Email, Password, Role, Blocked)
- Product (ID, Name, Category, Price, Stock)
- Cart / Cart Item (Product ID, Quantity)
- Order (ID, User ID, Date, Items, Total, Address, Status)
- Order Item (Product ID, Name, Quantity, Unit Price)
- Payment (ID, Method, Amount, Status)

Relationships:

- One customer can place many orders.
- One order contains one or more order items; each order item refers to one product.
- Each confirmed order has exactly one payment.
- The Admin is a special user whose role is “admin”.

3.5.4 Advantages of the Analysis Phase

- Clearly defines system requirements from the user's perspective.
- Identifies key actors, workflows and interactions.
- Helps in designing a modular, scalable and maintainable architecture.
- Serves as a reference for future development or enhancements.

3.5.5 Limitations Identified

- Data is stored in the browser, so it is not shared between different computers.
- The payment gateway is simulated and no real money is processed.
- Passwords are stored as plain text in the prototype and must be hashed on a server in production.
- The cart and login session are kept in memory and are lost when the page is refreshed.

4. SYSTEM DESIGN

System Design is the process of defining the overall architecture, components, modules, interfaces and data of a system so that it satisfies the specified requirements. For the Online Shopping System, this phase converts the results of system analysis into a blueprint that is practical and implementable for developers.

In this stage, the structure of the system is defined clearly: how users interact, how data flows between components, and how modules cooperate to achieve project goals such as cart handling, ordering and payment. System design is not algorithmic; it focuses on how the system works as a whole and how each part contributes to the final functionality.

4.1 Design Goals

1. **Scalability:** the system must support increasing numbers of users, products and orders; the admin can add new products at any time.
2. **Security:** role-based access control; admin functions are available only to the admin role; all text is escaped before display.
3. **Reliability:** an order is created only after successful payment, stock is never allowed to go below zero, and cancelling an order restores stock.
4. **Usability:** interfaces are intuitive, with clear messages (green for success, red for errors) and a consistent navigation bar.
5. **Maintainability:** the design allows future updates and additional modules without disturbing existing functionality.
6. **Efficiency:** screens update instantly after each action without reloading the page.
7. **Portability:** it runs on multiple devices and browsers with responsive layout and light/dark themes.

4.2 System Architecture

The Online Shopping System is designed using a three-tier architecture, ensuring modularity, scalability and efficient communication between the system components. The architecture is divided into three layers: Presentation Layer, Application Layer and Data Layer.

Presentation Layer	HTML, CSS and JavaScript user interface (customer and admin screens)
Application Layer	JavaScript logic: authentication, cart, order placement, payment and stock update (REST API in the server version)
Data Layer	Persistent storage of users, products, orders and payments (browser localStorage in the prototype, MySQL in production)

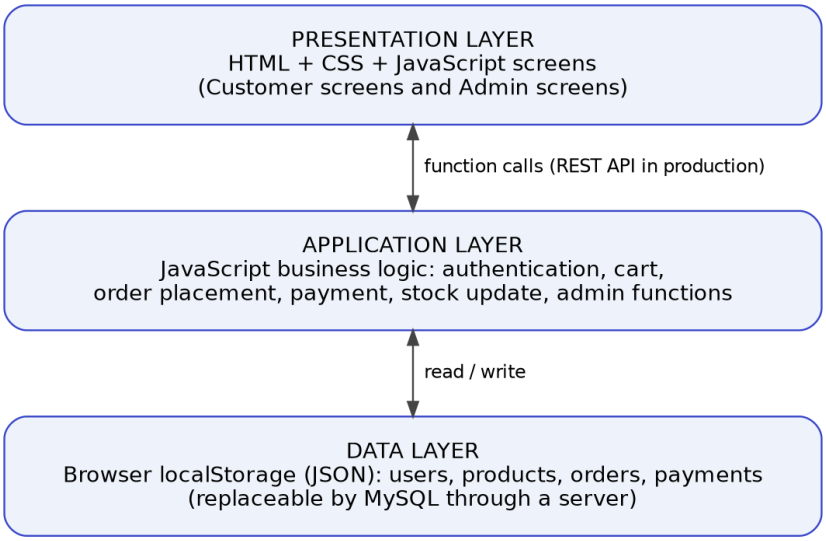


Fig. 4.2 System's Architecture Diagram

The **Presentation Layer** acts as the user interface and is developed using HTML, CSS and JavaScript. It provides different screens for customers and administrators. This layer captures user inputs, shows messages and updates the page after every action.

The **Application Layer** contains the system's core logic and business operations. It performs authentication, enforces the shopping rules (stock must be available, payment must succeed before an order is created, only confirmed orders can be cancelled by the customer), calculates totals and coordinates between the screens and the data store. Role-based access control ensures that only authorised users reach the right screens.

The **Data Layer** serves as the foundation for data management. It stores users, products, orders and payment records and guarantees that data stays consistent and persists between visits.

Together, these three layers create a robust and flexible system. This architectural design also makes the system easy to maintain, scale and enhance in future versions, for example by moving the application layer to a Node.js or Java server and the data layer to MySQL.

4.3 UML Diagrams

A UML diagram is a diagram based on the UML (Unified Modelling Language) with the purpose of visually representing a system along with its main actors, roles, actions, artifacts or classes, in order to better understand, alter, maintain or document information about the system.

UML is a modern approach to modelling and documenting software. It is based on diagrammatic representations of software components. As the old proverb says: “a picture is worth a thousand words”. By using visual representations, we are able to better understand possible flaws or errors in software or business processes. For the Online Shopping System the following diagrams were drawn in IBM Rational Software Architect Designer: the use case diagram, class diagram, activity diagram, sequence diagram, component diagram and deployment diagram.

4.4 Use Case Diagram

A Use Case Diagram represents the interactions between different users (actors) and the Online Shopping System. It shows how each role communicates with the system to perform specific functions.

Actors

- Customer – logs in, browses products, places orders, makes payments and logs out.
- Admin – logs in, manages products, orders and users, and logs out.

Major Use Cases

- Login / Logout
- Browse Product
- Place Order
- Make Payment
- Manage Product
- Manage Order
- Manage User

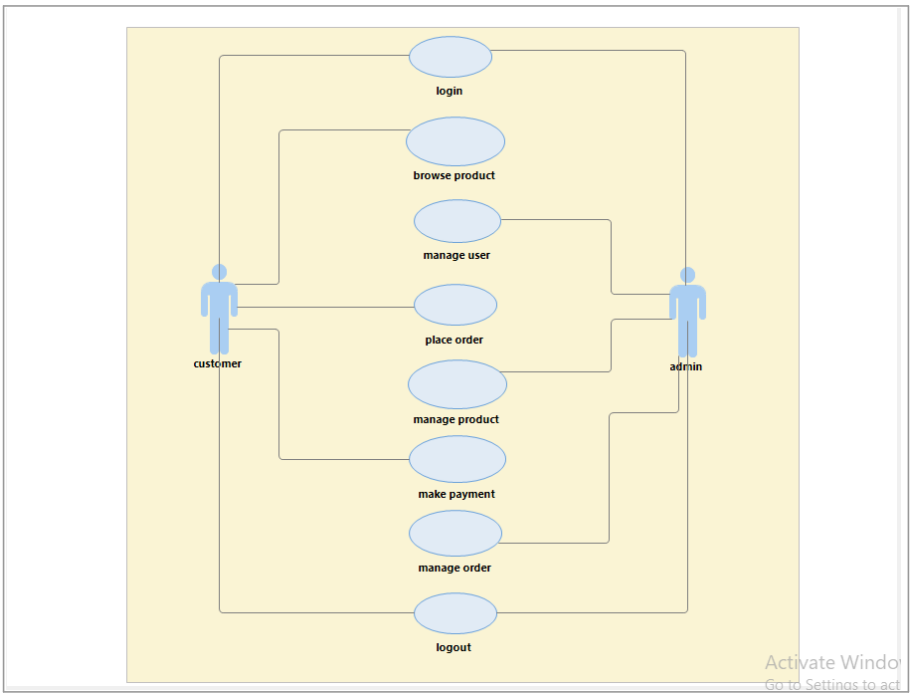


Fig. 4.4 Use Case Diagram

Purpose

The use case diagram helps to define the scope, functional requirements and user interactions of the Online Shopping System clearly and efficiently.

4.5 Class Diagram

A **Class Diagram** is a vital part of UML that illustrates the structure of the Online Shopping System by representing its classes, attributes, operations and the relationships among them. It provides a **static view** of the system, showing how data and functionalities are organised logically.

In the Online Shopping System, the class diagram helps developers understand the relationships between components such as **User, Order, Payment, Product and Admin**. Each class contains three main sections:

- **Class Name** – represents the entity (e.g., user, order).

- **Attributes** – defines the properties or data members (e.g., order date, product price).
- **Methods/Operations** – represents the behaviours or functions (e.g., place order(), make payment()).

Main Classes in Online Shopping System

- **user:** user ID, user name, email, password; login(), logout(), register()
- **order:** order ID, order date, status; place order(), cancel order()
- **payment:** payment ID, amount, payment method; make payment(), payment status()
- **product:** product ID, name, price, stock; update stock()
- **admin:** admin ID, name, email; add product(), remove product(), update product()

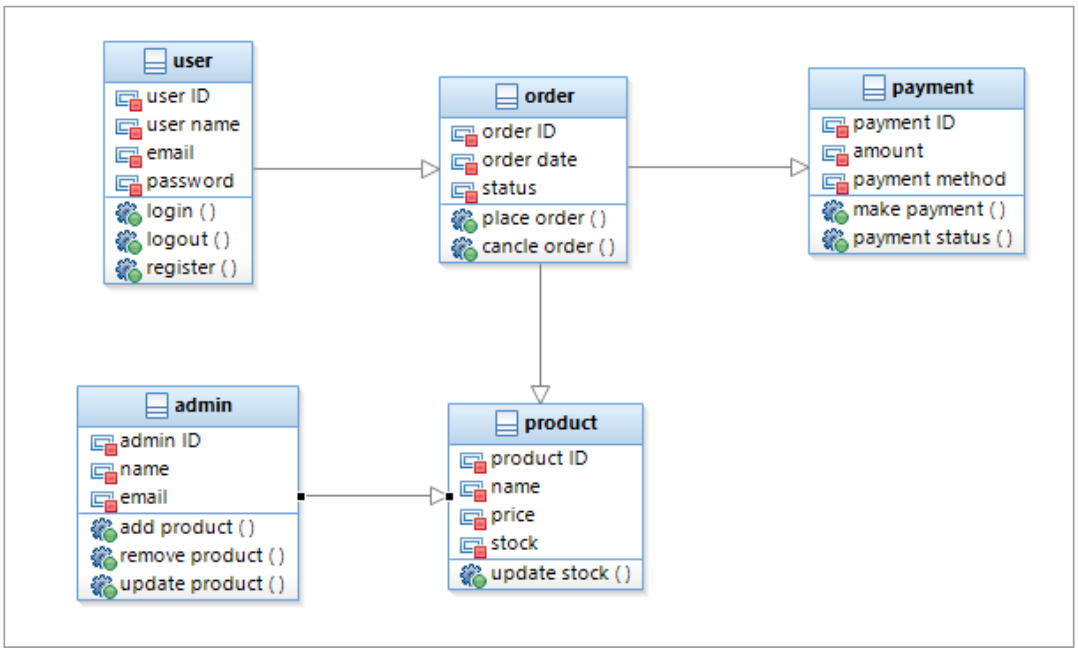


Fig. 4.5 Class Diagram

In the class diagram a user places orders, each order is paid through a payment and refers to the products that were bought, and the admin maintains the products. In the implementation the order items and the cart are stored as part of the order and the cart data (see Section 4.10).

4.6 Activity Diagram

An **Activity Diagram** in UML represents the flow of activities and decisions in the Online Shopping System. It helps visualise how a user interacts with the system and how processes move from one step to another.

In the Online Shopping System, it illustrates how a customer logs in or registers, browses and selects a product, decides whether to add it to the cart, proceeds to checkout, enters the shipping address and makes the payment. If the payment is not successful the customer returns to the payment step; when it is successful the order is successful and the activity ends.

Purpose:

- To represent the **workflow** of user interactions.
- To define the **sequence** and **conditions** of operations.
- To analyse and improve the **logical flow** of the system.

- To support **system understanding** during design and implementation.

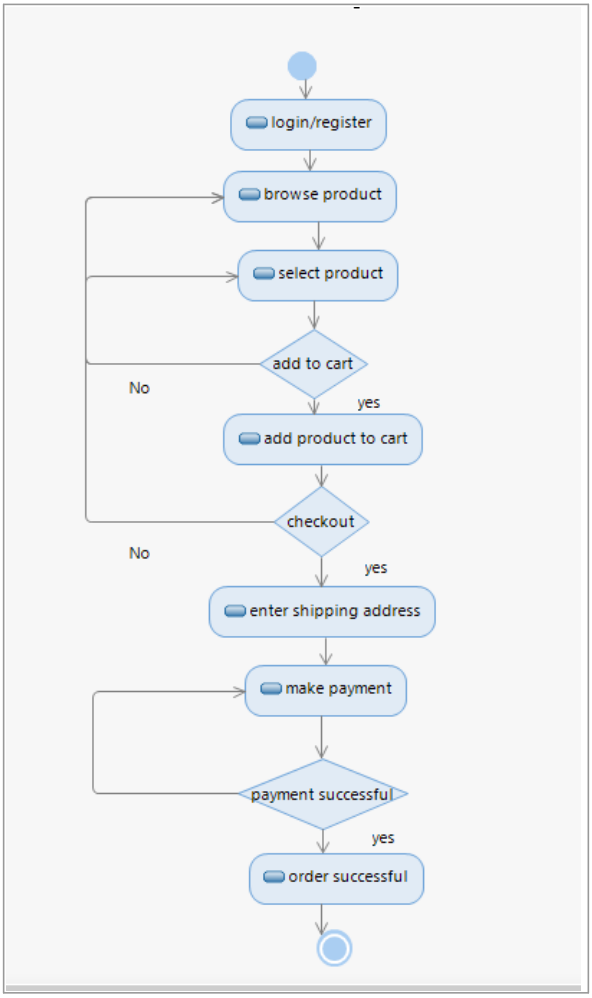


Fig. 4.6 Activity Diagram

Step	Activity	Decision / Result
1	Login / Register	Customer logs in or creates an account
2	Browse product, Select product	Product details and stock are shown
3	Add to cart?	No → return to browsing; Yes → product added to cart
4	Checkout?	No → return to browsing; Yes → enter shipping address
5	Make payment	Payment details are sent to the gateway
6	Payment successful?	No → return to make payment; Yes → order successful
7	Order successful	Order is saved and the activity ends

4.7 Sequence Diagram

A **Sequence Diagram** is a type of UML interaction diagram that shows how objects communicate with each other in a **specific order**. It represents the sequence of messages exchanged between users and system components to complete a process.

In the Online Shopping System, the sequence diagram illustrates searching for products, getting product information, adding to cart, checking out, creating the order, processing the payment and confirming the order.

Purpose:

- To represent the **detailed flow** of a use case.
- To model the **logic of system operations** step by step.
- To show how objects and components **interact** to complete a task.
- To help **plan, understand and document** the system's behaviour for existing or new features.

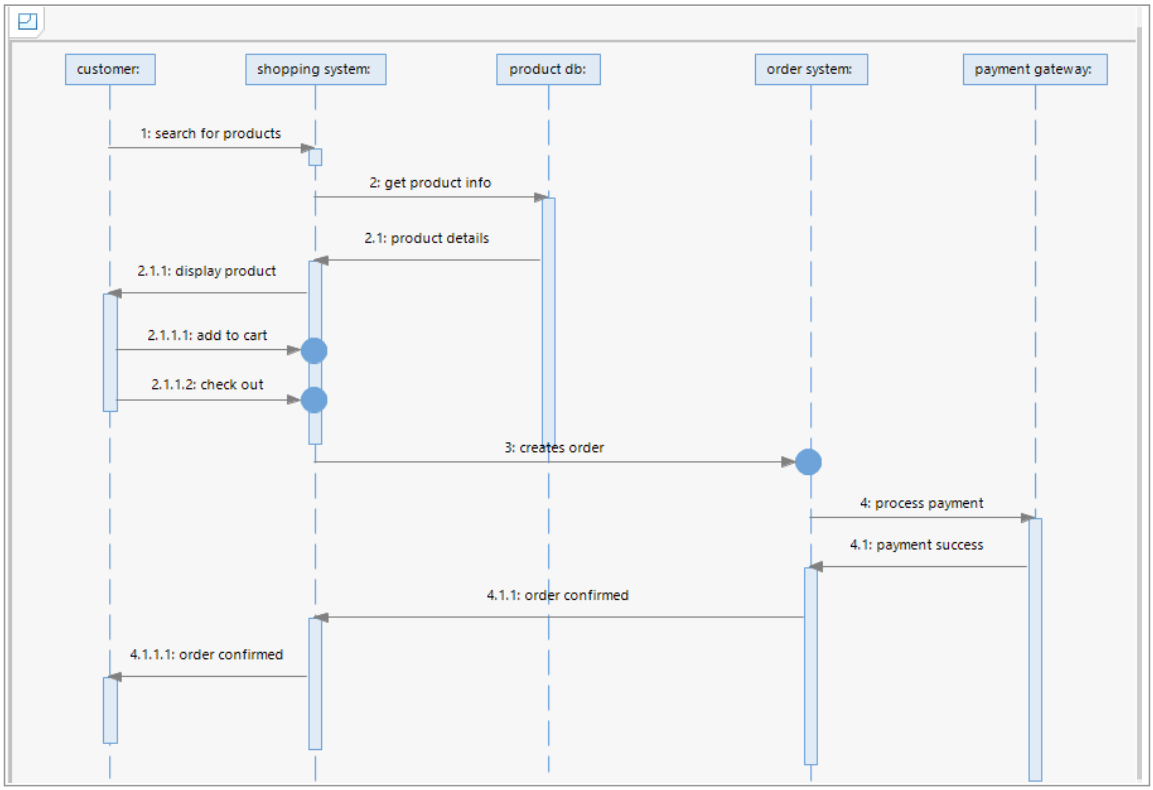


Fig. 4.7 Sequence Diagram

No.	Message	From → To
1	search for products	customer → shopping system
2	get product info	shopping system → product db
2.1	product details	product db → shopping system
2.1.1	display product	shopping system → customer
2.1.1.1	add to cart	customer → shopping system
2.1.1.2	check out	customer → shopping system
3	creates order	shopping system → order system
4	process payment	order system → payment gateway
4.1	payment success	payment gateway → order system
4.1.1	order confirmed	order system → shopping system
4.1.1.1	order confirmed	shopping system → customer

In the sequence diagram the customer first searches for products; the shopping system fetches details from the product database and displays them. The customer adds items and checks out; the shopping system creates an order in the order system, which asks the payment gateway to process the payment. On success, the confirmation travels back through the shopping system to the customer.

4.8 Component Diagram

The component diagram shows the physical organisation of the software. The **Customer Interface** (web / mobile app) and the **Admin Interface** (web app) communicate with the **Application layer**, which contains the User, Order, Product, Admin, Payment and Gateway components. The application layer stores users, products and orders in the **Database** component.

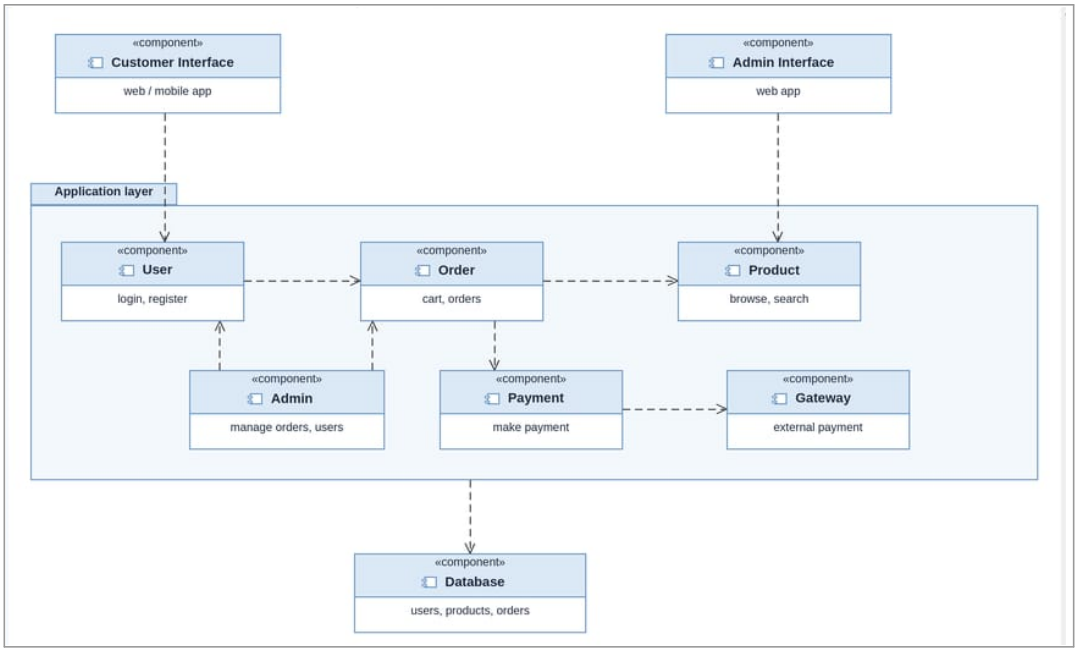


Fig. 4.8 Component Diagram

Component	Responsibility
Customer Interface	Web/mobile screens for registration, login, browsing, cart, checkout and orders
Admin Interface	Web screens for product, order and user management
User	Login and register
Order	Cart and orders
Product	Browse and search products
Admin	Manage orders and users
Payment	Make payment
Gateway	External payment processing
Database	Stores users, products and orders

4.9 Deployment Diagram

The deployment diagram shows how the software is placed on hardware nodes. The **Client Device** runs the client app and connects over HTTPS (port 443) to the **Application Server**, which hosts the shopping-app and payment-service containers. The application server connects through JDBC (port 3306) to the **Database Server** (MySQL) that holds the database schema, and over HTTPS (port 443) to the external **Payment Gateway** (payment-api).

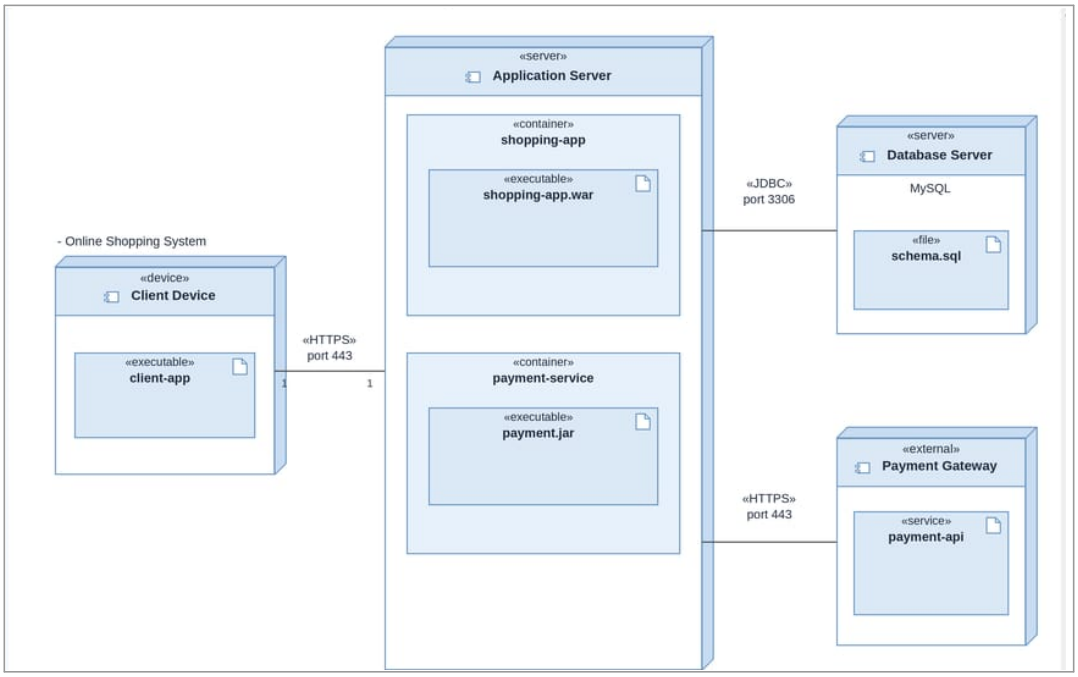


Fig. 4.9 Deployment Diagram

In the implemented prototype the client device is a web browser, the application layer is the JavaScript running in the browser, and the data is stored in the browser's localStorage. The same architecture can be moved to a server and a MySQL database by replacing the data-access functions.

4.10 Database Design

The class diagram was converted into three collections stored as one JSON document under the key **shoplite**. The admin class is implemented as a user whose role is “admin”, and the payment is stored inside the order.

Collection	Field	Type	Description
users	id, name, email	Number / String	Unique id, full name and unique email
	pw	String	Password (plain text in prototype; hash in production)
	role, blocked	String / Boolean	customer or admin; blocked flag
products	id, name, cat	Number / String	Product id, name and category
	price, stock	Number	Unit price in rupees and units in stock
	emoji	String	Icon shown on the product card
orders	id, userId, date	Number / String	Order id, owner and order date-time
	items[ ]	Array	Each item: id, name, qty, price
	total, address	Number / String	Order total and shipping address
	status	String	Confirmed, Packed, Shipped, Delivered or Cancelled
	payment	Object	id, method (Card/UPI), amount, status (Success)

5. IMPLEMENTATION

The Online Shopping System is implemented as a web application using HTML, CSS and JavaScript, with the browser's localStorage as the data store. The system is modular, with separate functions for Users, Products, Cart, Orders, Payment and Admin. Role-based screens ensure that customers and administrators see only their own functions, and every shopping rule is validated before data is saved. Each module was tested individually and together to ensure smooth operation.

5.1 Technology & Coding Approach

- **Frontend:** HTML5, CSS3 and vanilla JavaScript; the interface is a single-page application that updates the screen after every action without reloading.
- **Styling:** CSS variables with automatic light and dark mode and a responsive layout for phones and computers.
- **Data Handling:** JSON stored in localStorage with automatic creation and seeding of demo users and eight products on first run.
- **Authentication:** email and password check with role-based screens (customer or admin) and a blocked-user check.
- **Coding Approach:** modular and function-based, ensuring separation of concerns, easy maintenance and scalability.

5.2 Database & Information Handling

- **Database Used:** localStorage key *shoplite* (JSON) seeded with an admin account, a demo customer and 8 products in four categories (Electronics, Fashion, Home, Stationery). It can be replaced by MySQL as shown in the deployment diagram.
- **Information Handling:** users, products and orders are stored in separate collections; create, read, update and delete operations are implemented for products, orders and users; input is validated before storage; and stock is updated together with the order so that the two never disagree.

Cart total calculation. The cart total is the sum of unit price multiplied by quantity for every item in the cart:

```
total = sum( product.price * item.qty )   for every item in cart
```

For example, two T-shirts at Rs. 499 and one Notebook Set at Rs. 249 give a total of Rs. 1,247.

5.3 Programming Style

- **Function-based:** each feature (register, login, cart, payment, order, admin) is implemented as a separate function.
- **Reusable Functions:** common tasks such as showing messages (flash), escaping text (esc), formatting money (inr) and finding a product (prod) are written once and reused.
- **Error Handling:** every rule violation shows a clear message (for example “Only 12 of Smart Watch in stock.”).
- **Code Documentation:** sections are labelled with comments for easier understanding during future updates.

5.4 Verification and Validation

- **Verification:** ensures the system is built according to the design specifications: each UML class, use case and activity step was mapped to a function or screen; automated tests were written for the application logic.
- **Validation:** ensures the system fulfils the intended functionality for each user role; test cases were created for login, cart, checkout, payment, order cancellation and admin management.

5.5 Implementation of Platform

The platform is divided into modular components, each handling specific functionalities:

1. **User Authentication Module:** registration, login, logout and role-based screens.
2. **Product Module:** product catalogue with search and category filter; admin add, edit and remove.
3. **Cart Module:** add, change quantity, remove and total with stock limits.
4. **Order Module:** order creation, stock reduction, cancellation and status updates.
5. **Payment Module:** card or UPI details, simulated gateway, success and decline handling.
6. **Admin Module:** product, order and user management screens.

Project Structure

File / Folder	Purpose
index.html	Page layout (navigation bar, message area and main content area)
style.css	Styling of the customer and admin screens with light/dark themes
app.js	All application logic: authentication, cart, orders, payment and admin functions
test_shoplite.js	Automated test script (Node.js) for the application logic

Function Summary

Function	Purpose	Access
register(), login(), logout()	Create account, log in (rejects wrong credentials and blocked users), log out	Public / User
add(id), qty(id, d)	Add to cart and change quantity within stock	Customer
checkout()	Requires login; blocks admin; opens checkout	Customer
pay()	Validates address and payment, simulates gateway, creates order and reduces stock	Customer
cancel(id)	Cancels a confirmed order and restores stock	Customer
addProd(), upd(), delProd()	Add, update price/stock and remove products	Admin
setStatus(id, s)	Change order status (stock restored on cancellation)	Admin
block(id)	Block or unblock a user	Admin
grid(), setQ(), setCat()	Show products, search and category filter	Customer

How to run the project

- Keep **index.html**, **style.css** and **app.js** together in one folder.
- Open the folder in Visual Studio Code and install the **Live Server** extension.

- Right-click **index.html** and choose **Open with Live Server** (or simply double-click index.html).
- Admin login: admin@shop.com / admin123. Customer login: demo@shop.com / demo123.
- To run the automated tests: install Node.js and run **node test_shoplite.js** in the same folder.

5.5.1 Order Placement and Stock Algorithm

The most important rule of the system is that an order is created only after a successful payment and that stock is never oversold. The system follows the steps below every time a customer pays. If any check fails, a clear message is shown and nothing is saved.

1. Check that the shipping address has at least 8 characters -> else “Enter a full shipping address.”
2. Check that the card number or UPI ID is entered -> else “Enter your card number / UPI ID.”
3. For every cart item check that quantity <= current stock -> else “Stock changed for the product. Update your cart.”
4. Send the payment to the gateway (simulated delay of about one second).
5. If the gateway declines the payment (card number ending in 0000) -> show “Payment declined by the gateway. Your order was not placed.” and keep the cart and address.
6. Create the order with items, total, address, status *Confirmed* and the payment record.
7. Reduce the stock of every product by the ordered quantity, save the data and empty the cart.

When the customer cancels a confirmed order, or the admin sets an order to Cancelled, the order quantities are added back to stock so that the stock stays correct.

5.5.2 Security Implementation

- **Role-based access:** admin screens (Products, Orders, Users) are shown only to the admin, and customer screens are redirected to login when the user is not logged in; an admin cannot check out.
- **Blocked users:** an administrator can block a customer; a blocked customer cannot log in.
- **Input validation:** the system checks every input (empty values, short passwords, duplicate emails, negative prices or stock, short addresses) and escapes text before it is displayed to avoid script injection.
- **Production hardening (future):** passwords are stored as plain text in this prototype; in the server version they must be hashed with a salted algorithm (for example bcrypt or scrypt) and sessions protected with tokens over HTTPS.

5.5.3 Error Handling

Every rule violation shows a readable message at the top of the page in red. The most common cases are listed below.

Situation	Message shown
Wrong email or password	Email or password is wrong. Try again.
Blocked account	This account is blocked. Contact the admin.
Duplicate email	That email is already registered. Log in instead.
Weak password / missing details	Enter a name, a valid email and a password of 4+ characters.
Adding more than stock	Only N of the product in stock.
Checkout without login	Log in to check out.

Situation	Message shown
Admin tries to check out	Admins cannot place orders. Use a customer account.
Payment declined	Payment declined by the gateway. Your order was not placed. Try another payment method.
Invalid product values	Enter a name, a price above 0 and a stock of 0 or more.

6. TESTING

Testing is an essential phase in software development to ensure that the Online Shopping System is reliable, functional and user-friendly. It helps in detecting errors, verifying requirements and improving system quality before deployment.

6.1 Testing Activities :

Unit Testing: Each module is tested individually to ensure that the code performs as expected. For example:

- Cart Module: check adding, quantity change, stock limit and total.
- Payment Module: check declined and successful payments.
- Product Module: check add, update, remove and validation.

Integration Testing: Verifies that all modules work together smoothly. For instance:

- A successful payment creates the order, reduces stock and empties the cart.
- Cancelling an order restores stock and the product list shows the restored quantity.

System Testing: The complete system is tested as a whole to ensure it meets functional and security requirements. The application logic was tested with a Node.js script and the screens were run in the Google Chrome browser.

User Acceptance Testing (UAT): Actual users (a customer and an admin) test the system to ensure it satisfies their requirements and is easy to use.

6.2 Testing Types :

- **Functional Testing:** confirms that all features like registration, login, cart, checkout, payment, order cancellation and admin management work correctly.
- **Usability Testing:** ensures the system is easy to navigate, responsive and user-friendly for all roles.
- **Security Testing:** verifies authentication, role-based access and blocked-user handling.
- **Validation Testing:** checks duplicate entries, stock limits and invalid inputs.
- **Compatibility Testing:** ensures the system works across different devices and browsers.

6.3 Test Cases:

The test cases below were executed with an automated script (test_shoplite.js, Node.js) against the application logic of the system, and the screens were verified in the browser (Chapter 7).

Test Case	Input	Expected Output	Result
Register	Valid name, email, password	Account created, user logged in	Pass
Duplicate registration	Same email again	Error: email already registered	Pass
Weak password	Password of 2 characters	Error: 4+ characters required	Pass
Admin login	admin@shop.com / admin123	Login successful, admin screens	Pass
Invalid login	Wrong password	Error: email or password is wrong	Pass
Customer login	test@mail.com / secret1	Login successful, shop screen	Pass

Test Case	Input	Expected Output	Result
Add to cart	Add Wireless Earbuds twice	Cart quantity 2	Pass
Change quantity	Reduce quantity by 1	Quantity becomes 1	Pass
Remove from cart	Quantity reduced to 0	Item removed from cart	Pass
Stock limit	Add Smart Watch 30 times (stock 12)	Error: only 12 in stock	Pass
Cart total	2 T-shirts (499) + 1 Notebook Set (249)	Total Rs. 1247	Pass
Checkout without login	Check out while logged out	Redirected to login	Pass
Admin checkout	Admin tries to check out	Error: admins cannot place orders	Pass
Short address	Address of 5 characters	Error: enter a full shipping address	Pass
Declined payment	Card number ending 0000	Payment declined, no order, stock unchanged	Pass
Empty payment detail	No card number	Error: enter card number	Pass
Successful payment	Valid card, 2 T-shirts + 1 Notebook Set	Order created, cart emptied	Pass
Stock reduced	After successful order	T-shirt stock 60 to 58, Notebook 100 to 99	Pass
Order record	Order status and payment	Status Confirmed, payment Success, total 1247	Pass
Cancel order	Customer cancels confirmed order	Status Cancelled, stock restored	Pass
UPI payment	UPI ID test@upi	Order confirmed with UPI	Pass
Add product	Admin adds Test Mug, Rs. 150, stock 10	New product appears in catalogue	Pass
Invalid product	Empty name and price 0	Error: enter a name and price above 0	Pass
Update price and stock	Earbuds price 1599, stock 30	Values updated	Pass
Negative value	Price -5	Error: enter a number of 0 or more; price unchanged	Pass
Update order status	Admin sets order to Delivered	Status Delivered	Pass
Block user	Admin blocks test@mail.com	User blocked	Pass
Blocked user login	Blocked user logs in	Error: account is blocked	Pass
Remove product	Admin removes Test Mug	Product removed	Pass
Search	Search text: shirt	Only Cotton T-Shirt listed	Pass
Category filter	Category: Home	Only Home products listed	Pass

Test summary: 31 test cases were executed, 31 passed and 0 failed. Testing ensures that all modules of the Online Shopping System work correctly and provide a smooth experience for customers and administrators.

7. RESULTS & DEMONSTRATION

7.1 User Registration and Login

The first screen of the system is the shop page. A new customer opens the Log in page, enters name, email and password in the registration form and creates an account. Existing customers and the admin log in with their email and password and are taken to the screen of their role.

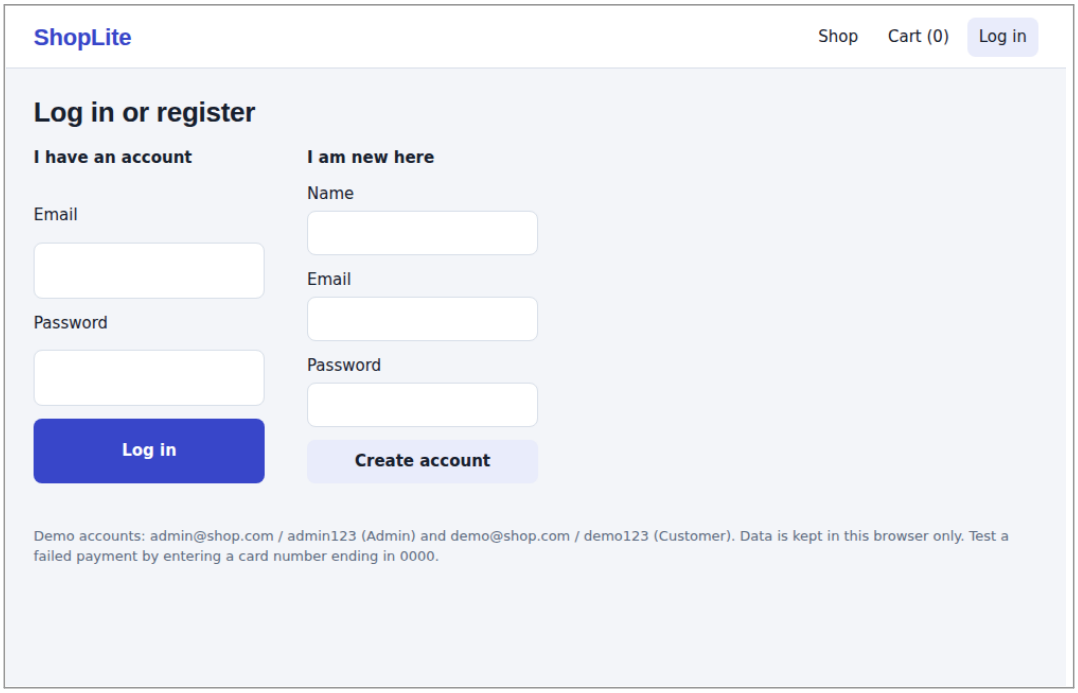


Fig. 7.1 (a) Login and Registration Page

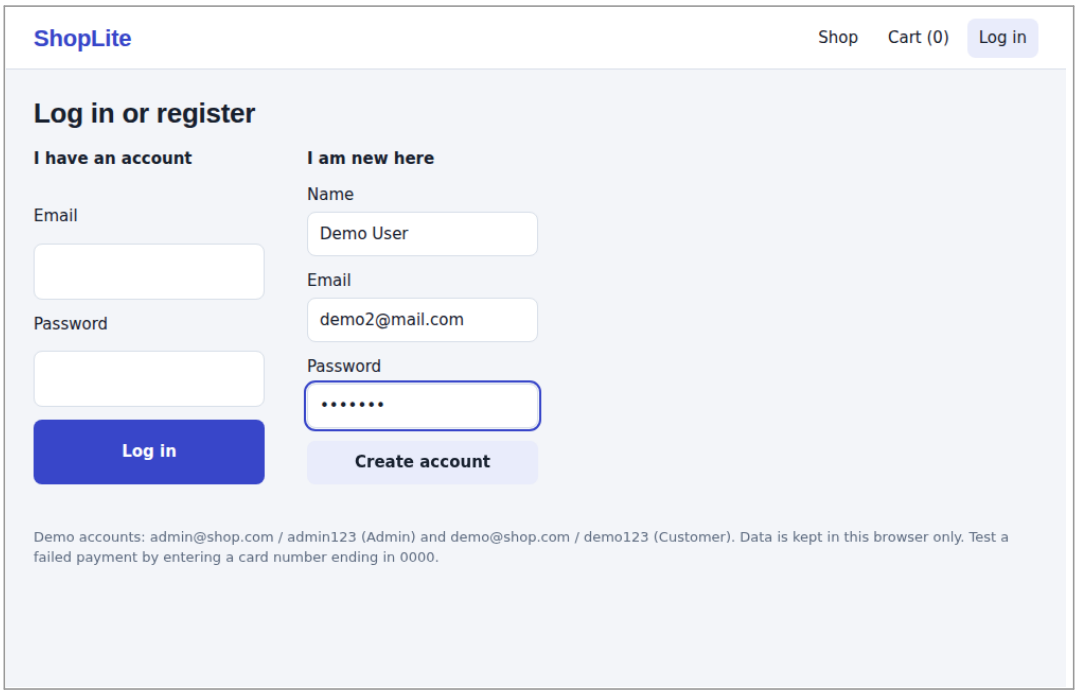


Fig. 7.1 (b) User Registration

7.2 Browsing Products

After login the customer sees the product catalogue. Each card shows the product icon, name, category, stock and price. The customer can search by name, filter by category and click Add to cart.

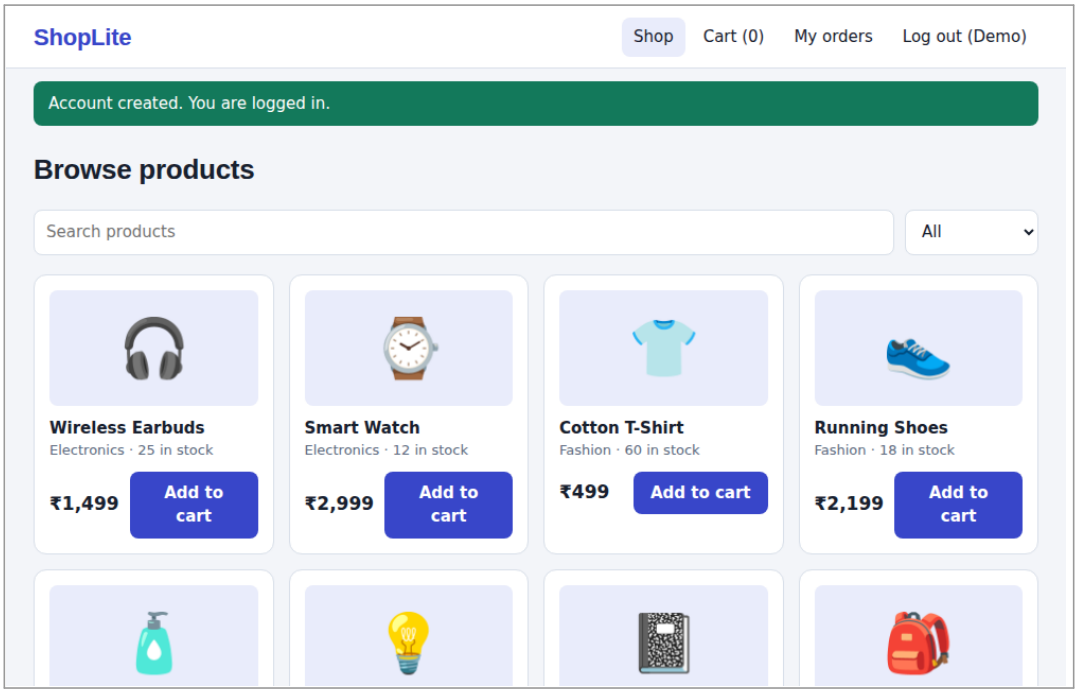


Fig. 7.2 Product Catalogue

7.3 Cart

Items added to the cart are listed with price, quantity controls and subtotal. The total is calculated automatically and the quantity cannot exceed the available stock.

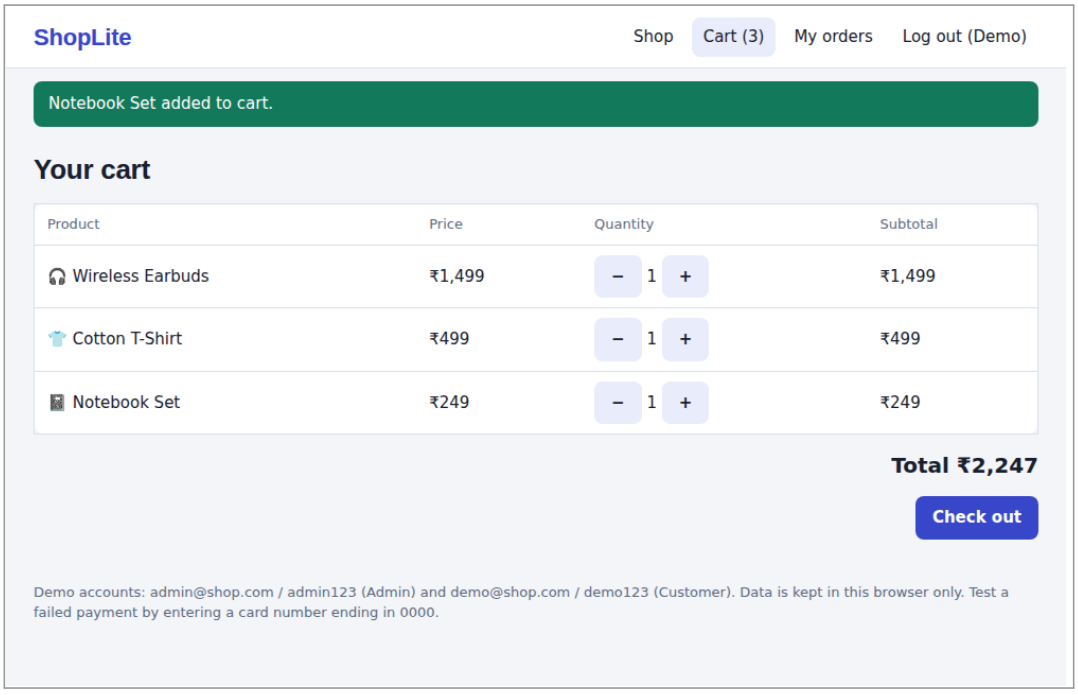


Fig. 7.3 Shopping Cart

7.4 Checkout, Payment and Order

When the customer clicks Check out, the checkout page asks for the shipping address and payment method (card or UPI). If the gateway declines the payment, an error is shown and no order is created; the address and cart are kept so that the customer can retry.

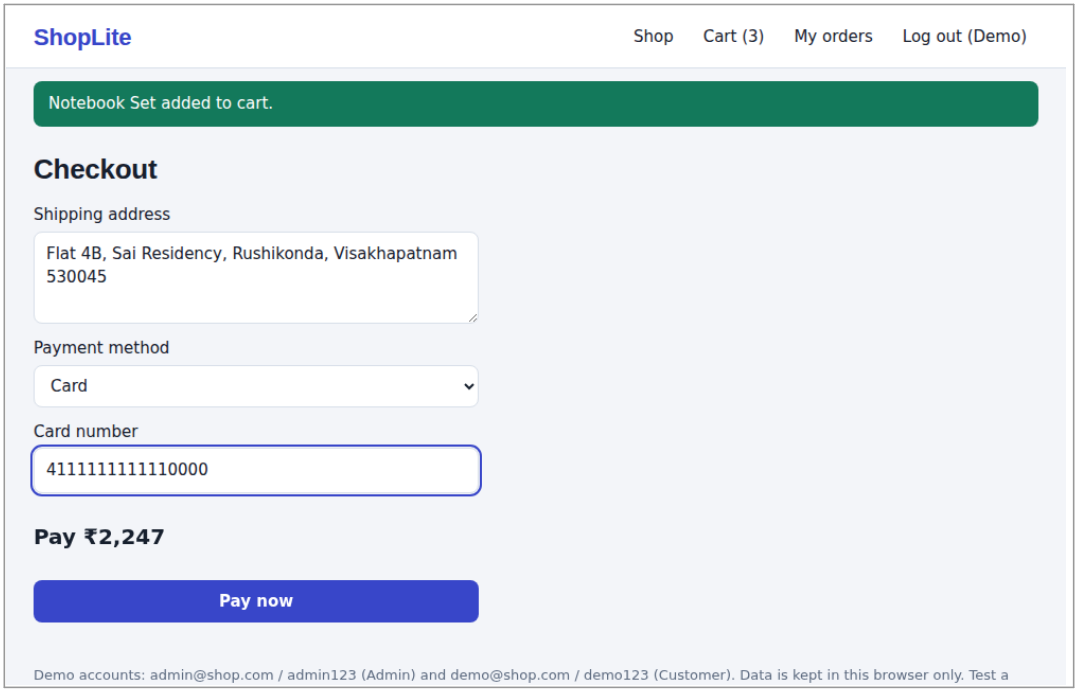


Fig. 7.4 (a) Checkout Page

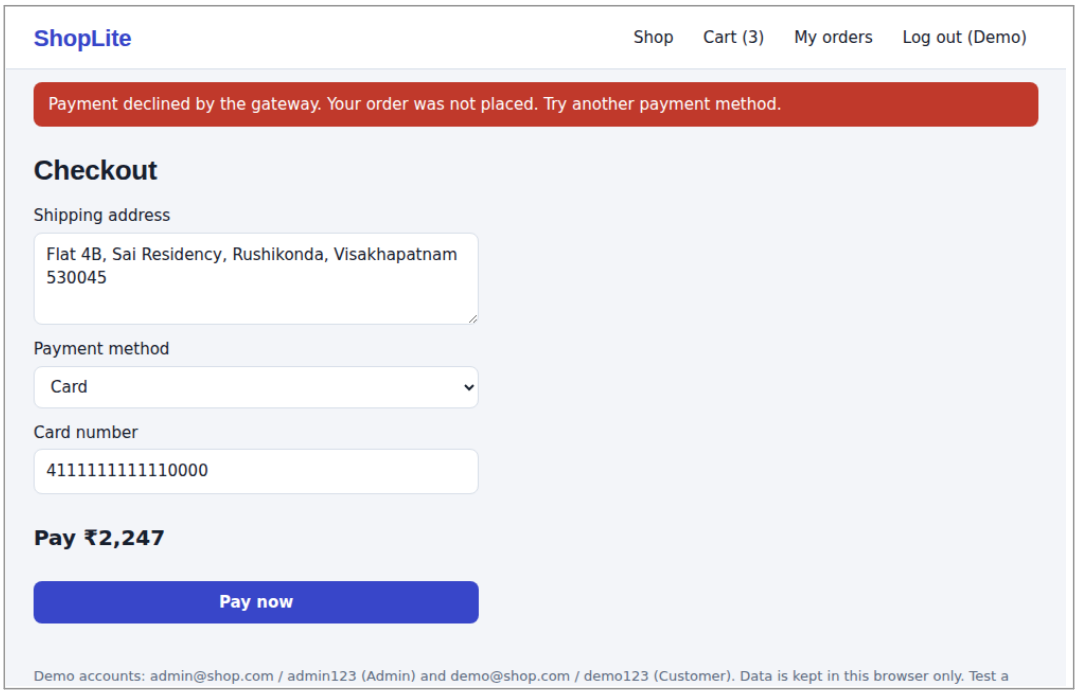


Fig. 7.4 (b) Payment Declined – No Order Created

After a successful payment the order is confirmed, stock is reduced and the order appears under My orders with a Cancel order option while it is still confirmed.

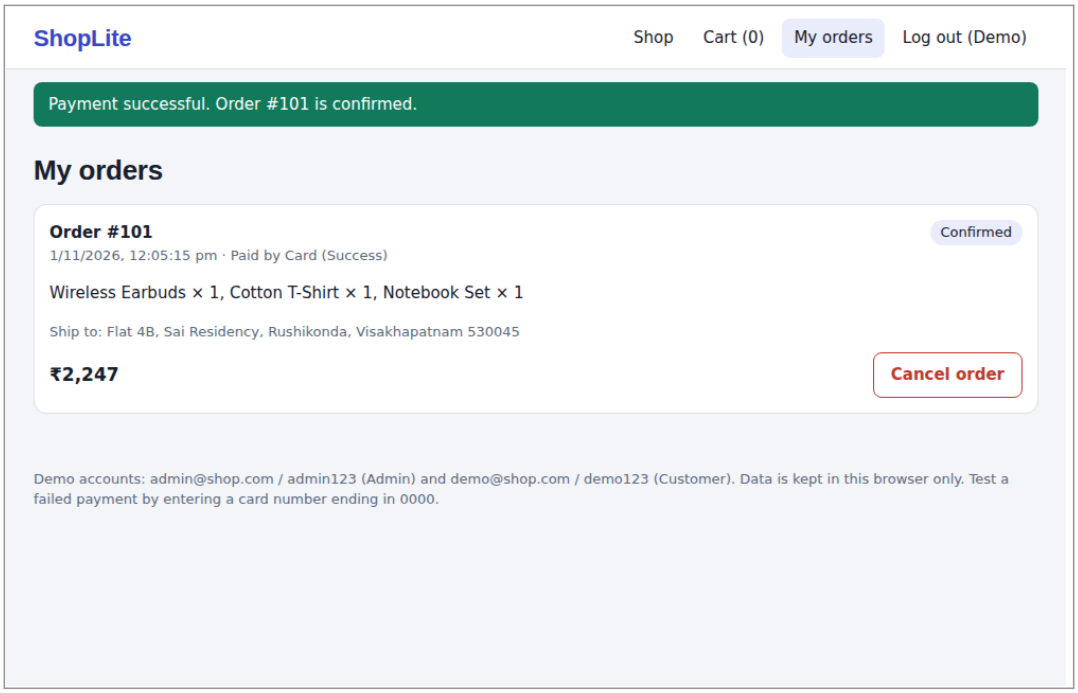


Fig. 7.4 (c) Order Confirmed

7.5 Admin Screens

The admin sees Products, Orders and Users in the navigation bar. In Products the admin can add a new product, edit price and stock and remove products. The system validates admin actions; for example, adding a product without a name or price shows an error.

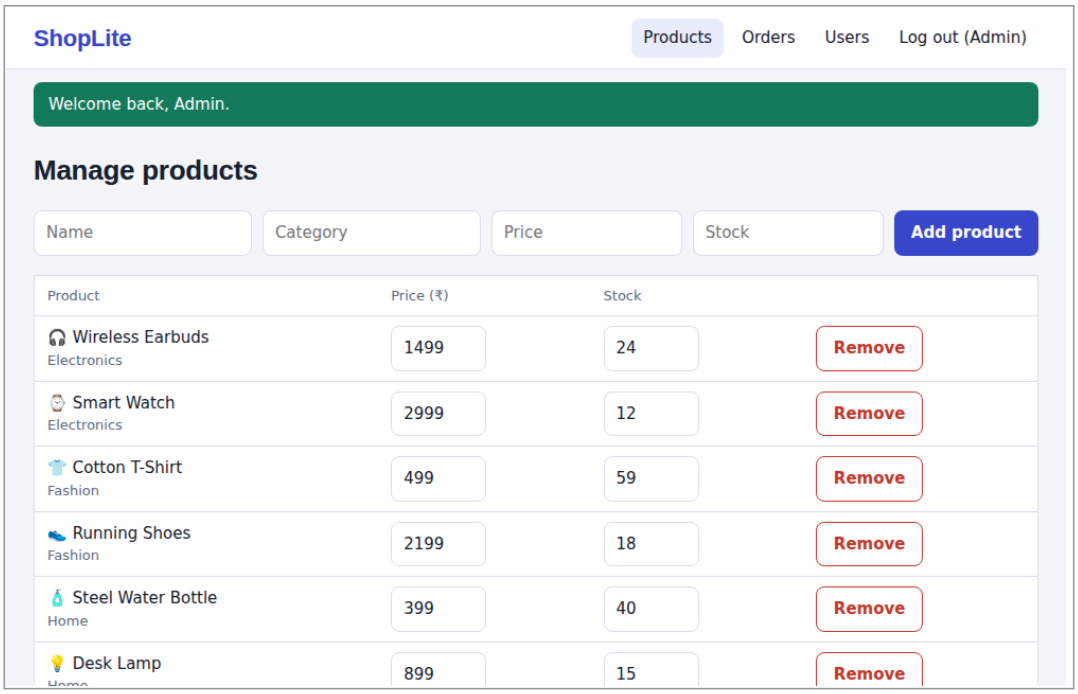


Fig. 7.5 (a) Admin – Manage Products

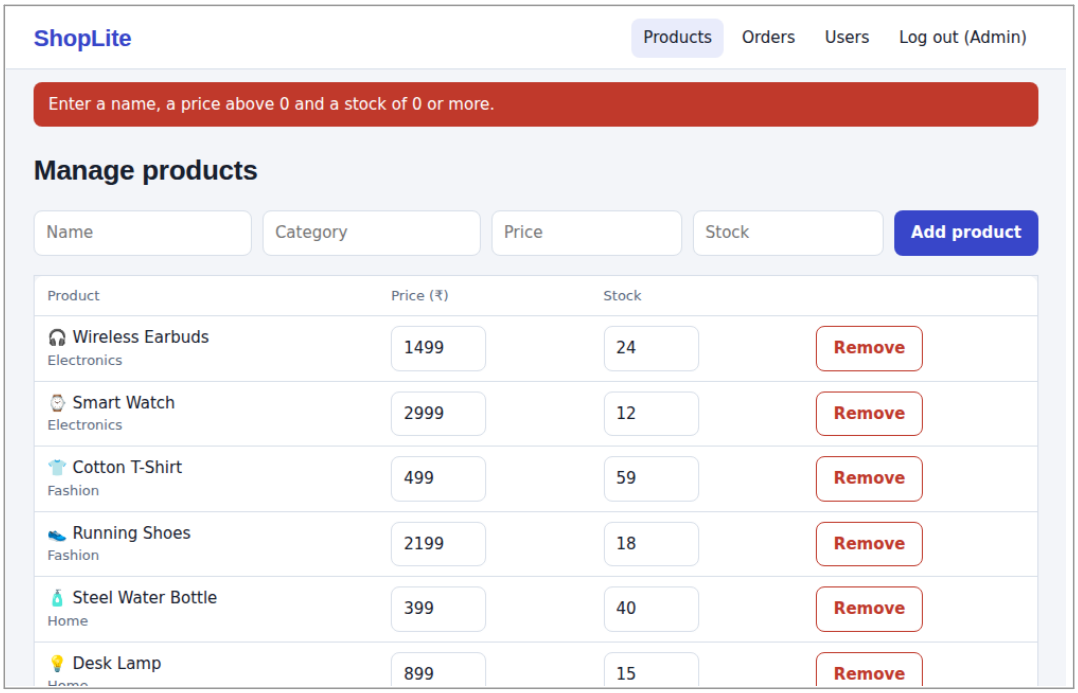


Fig. 7.5 (b) Validation Message for Invalid Product

In Orders the admin can see all orders with customer, items and total and change the status of each order. In Users the admin can block or unblock customers.

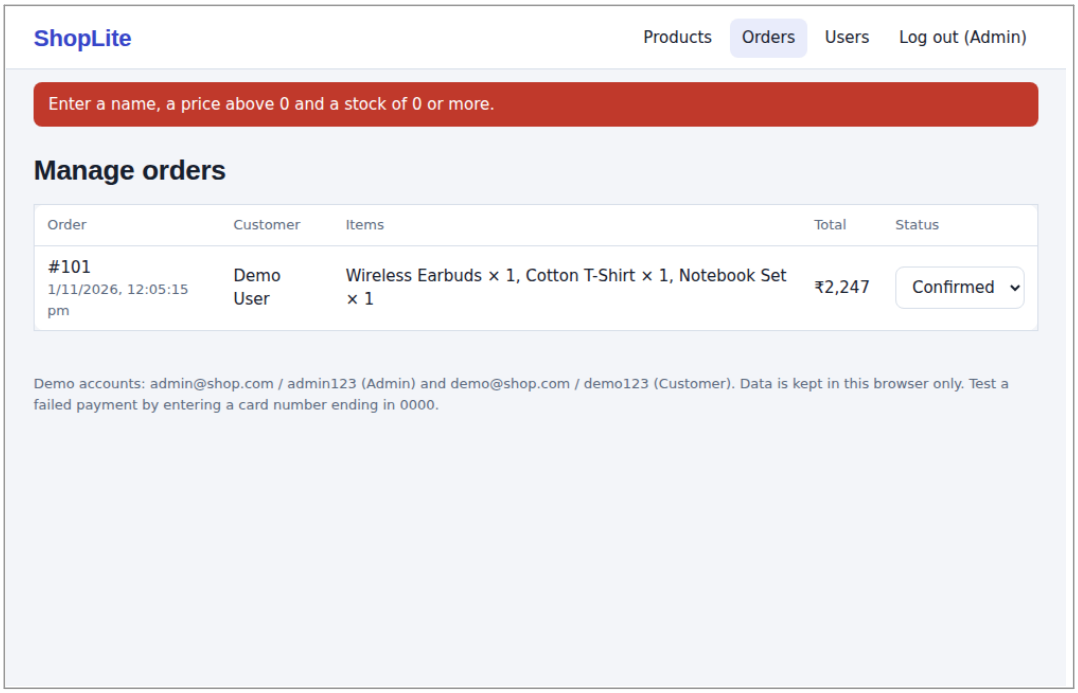


Fig. 7.5 (c) Admin – Manage Orders

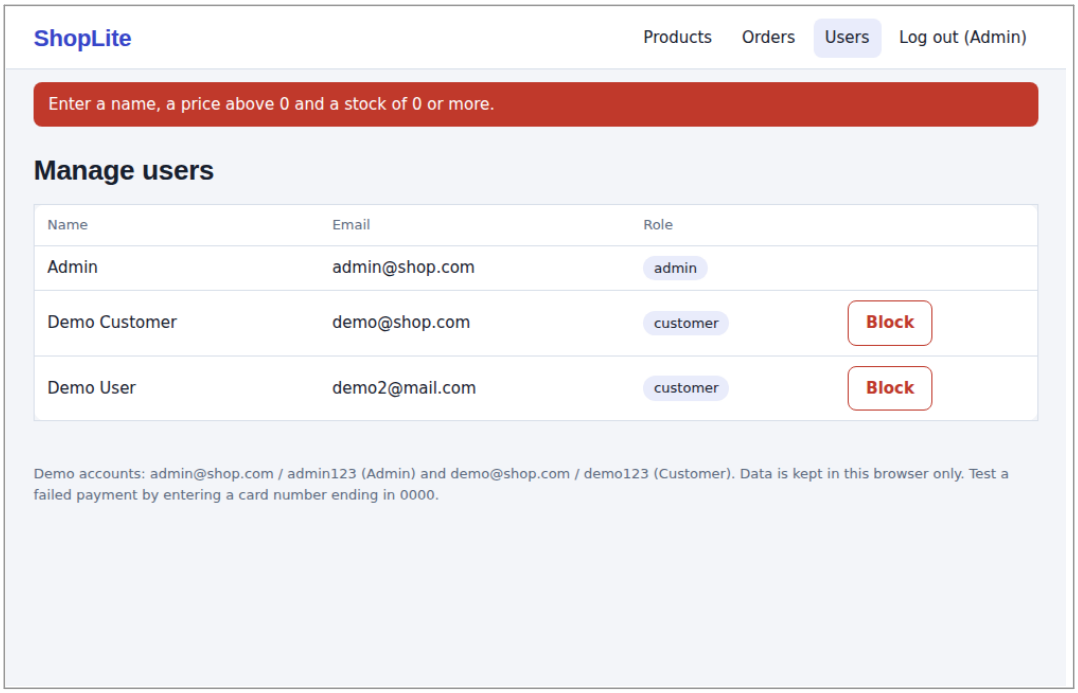


Fig. 7.5 (d) Admin – Manage Users

8. SAMPLE CODE

8.1 Frontend & Implementation

index.html

```
<!doctype html>
<html lang="en">
<head>
<meta charset="utf-8">
<title>ShopLite — Online Shopping System</title>
<meta name="viewport" content="width=device-width, initial-scale=1, viewport-fit=cover">
<link href="https://fonts.googleapis.com/css2?family=Bricolage+Grotesque:wght@600;700&family=DM+Sans:wght@400;50
    0;700&display=swap" rel="stylesheet">
<link rel="stylesheet" href="style.css">
</head>
<body>
<header><nav id="nav"></nav></header>
<main><div id="msg"></div><div id="app"></div></main>
<footer>Demo accounts: admin@shop.com / admin123 (Admin) and demo@shop.com / demo123 (Customer). Data is kept
    in this browser only. Test a failed payment by entering a card number ending in 0000.</footer>
<script src="app.js"></script>
</body>
</html>
```

style.css

```
:root{--bg:#f3f5f9;--sf:#fff;--ink:#17202e;--mu:#5d6b80;--ac:#3846c9;--acx:#fff;--tint:#e9ecfb;--ln:#d9e0ea;--ok
    :#13795b;--bad:#c0392b;box-sizing:border-box;padding-top:env(safe-area-inset-top,0px);padding-bottom:env(saf
    e-area-inset-bottom,0px)}
html{scroll-padding-top:env(safe-area-inset-top,0px)}
@media (prefers-color-scheme:dark){:root:not([data-theme="light"]){--bg:#10151e;--sf:#18202c;--ink:#e8edf5;--mu:
    #97a4b8;--ac:#8c97f5;--acx:#10151e;--tint:#222b3d;--ln:#2a3545;--ok:#46c79c;--bad:#ff8a7a;color-scheme:dark}
    }
:root[data-theme="dark"]{--bg:#10151e;--sf:#18202c;--ink:#e8edf5;--mu:#97a4b8;--ac:#8c97f5;--acx:#10151e;--tint:
    #222b3d;--ln:#2a3545;--ok:#46c79c;--bad:#ff8a7a;color-scheme:dark}
*{box-sizing:border-box}
body{margin:0;background:var(--bg);color:var(--ink);font:15px/1.5 "DM Sans",system-ui,sans-serif}
h1,h2,.brand{font-family:"Bricolage Grotesque","DM Sans",sans-serif;letter-spacing:-.01em}
h2{font-size:26px;margin:22px 0 12px}
header{background:var(--sf);border-bottom:1px solid var(--ln)}
nav{max-width:980px;margin:auto;padding:10px 16px;display:flex;flex-wrap:wrap;align-items:center;gap:6px}
.brand{font-size:22px;font-weight:700;margin-right:auto;color:var(--ac)}
nav button{background:none;border:0;color:var(--ink);font:inherit;padding:7px
    11px;border-radius:8px;cursor:pointer}
nav button:hover,nav button.on{background:var(--tint)}
main{max-width:980px;margin:auto;padding:0 16px 40px}
.bar{display:flex;gap:10px;flex-wrap:wrap;margin:18px 0}
input,select,textarea{font:inherit;color:var(--ink);background:var(--sf);border:1px solid
    var(--ln);border-radius:8px;padding:9px 11px;max-width:100%}
input:focus,select:focus,textarea:focus,button:focus-visible{outline:2px solid var(--ac);outline-offset:1px}
.bar input{flex:1;min-width:180px}
.btn{background:var(--ac);color:var(--acx);border:0;border-radius:8px;padding:9px
    16px;font:inherit;font-weight:700;cursor:pointer}
.btn.ghost{background:var(--tint);color:var(--ink)}
.btn.bad{background:none;color:var(--bad);border:1px solid var(--bad)}
.btn:disabled{opacity:.5;cursor:default}
.grid{display:grid;grid-template-columns:repeat(auto-fill,minmax(200px,1fr));gap:14px}
.card{background:var(--sf);border:1px solid var(--ln);border-radius:12px;padding:14px}
.em{font-size:54px;text-align:center;background:var(--tint);border-radius:8px;padding:14px 0;margin-bottom:10px}
.price{font-weight:700;font-size:17px}
.mu{color:var(--mu);font-size:13px}
.row{display:flex;justify-content:space-between;align-items:center;gap:10px;margin-top:10px}
.wrap{overflow-x:auto}
table{width:100%;border-collapse:collapse;background:var(--sf);border:1px solid var(--ln);border-radius:12px}
th,td{padding:9px 12px;text-align:left;border-bottom:1px solid var(--ln);vertical-align:middle}
th{font-size:13px;color:var(--mu);font-weight:500}
td input{width:90px}
.tag{display:inline-block;padding:2px 10px;border-radius:99px;background:var(--tint);font-size:13px}
.form{max-width:420px;display:grid;gap:12px}
.form label{display:grid;gap:4px;font-weight:500}
#msg{position:sticky;top:env(safe-area-inset-top,0px);z-index:5}
#msg div{margin:12px 0 0;padding:10px 14px;border-radius:8px;background:var(--ok);color:var(--acx);font-weight:5
    00}
#msg div.err{background:var(--bad)}
.tot{font-size:20px;font-weight:700;text-align:right;margin:14px 0}
footer{max-width:980px;margin:auto;padding:0 16px 30px;color:var(--mu);font-size:13px}
```

8.2 Application Logic

app.js

```
const $=s=>document.querySelector(s);
const esc=s=>String(s).replace(/[&<>"]/g,c=>({"&":"&amp;","<":"&lt;",">":"&gt;",'"':"&quot;"}[c]));
const inr=n=>"₹"+Number(n).toLocaleString("en-IN");
const seed=()=>({nid:100,
 users:[{id:1,name:"Admin",email:"admin@shop.com",pw:"admin123",role:"admin",blocked:false},{id:2,name:"Demo
    Customer",email:"demo@shop.com",pw:"demo123",role:"customer",blocked:false}],
 products:[[1,"Wireless Earbuds","Electronics",1499,25,"*"],[2,"Smart
    Watch","Electronics",2999,12,"□"],[3,"Cotton T-Shirt","Fashion",499,60,"*"],[4,"Running
    Shoes","Fashion",2199,18,"*"],[5,"Steel Water Bottle","Home",399,40,"*"],[6,"Desk
    Lamp","Home",899,15,"*"],[7,"Notebook Set","Stationery",249,100,"*"],[8,"Backpack","Fashion",1299,20,"*"]].m
    ap(p=>({id:p[0],name:p[1],cat:p[2],price:p[3],stock:p[4],emoji:p[5]})),
 orders:[]});
let db,me=null,view="shop",cart=[],q="",cat="All",busy=false;
try{db=JSON.parse(localStorage.getItem("shoplite"))}catch(e){}
if(!db||!db.users)db=seed();
const save=()=>{try{localStorage.setItem("shoplite",JSON.stringify(db))}catch(e){}};
const prod=id=>db.products.find(p=>p.id==id);
function flash(t,err){$("#msg").innerHTML='<div class="'+(err?"err":"")+'">'+esc(t)+"</div>";clearTimeout(flash.
    t);flash.t=setTimeout(()=>$("#msg").innerHTML="",3800)}
function go(v){view=v;render()}

/* ---------- User: register, login, logout ---------- */
function login(){const u=db.users.find(x=>x.email==$("#em").value.trim().toLowerCase()&&x.pw==$("#pw").value);
 if(!u)return flash("Email or password is wrong. Try again.",1);
 if(u.blocked)return flash("This account is blocked. Contact the admin.",1);
 me=u;flash("Welcome back, "+u.name+".");go(u.role=="admin"?"aprod":(cart.length?"cart":"shop"))}
function register(){const n=$("#rn").value.trim(),e=$("#re").value.trim().toLowerCase(),p=$("#rp").value;
 if(!n||!e.includes("@")||p.length<4)return flash("Enter a name, a valid email and a password of 4+
    characters.",1);
 if(db.users.some(x=>x.email==e))return flash("That email is already registered. Log in instead.",1);
 const u={id:db.nid++,name:n,email:e,pw:p,role:"customer",blocked:false};db.users.push(u);save();me=u;flash("Acc
    ount created. You are logged in.");go("shop")}
function logout(){me=null;cart=[];flash("You are logged out.");go("shop")}

/* ---------- Cart ---------- */
function add(id){const p=prod(id),c=cart.find(x=>x.id==id),n=c?c.qty:0;
 if(n>=p.stock)return flash("Only "+p.stock+" of "+p.name+" in stock.",1);
 c?c.qty++:cart.push({id:id,qty:1});flash(p.name+" added to cart.");nav()}
function qty(id,d){const c=cart.find(x=>x.id==id),p=prod(id);c.qty+=d;
 if(c.qty>p.stock){c.qty=p.stock;flash("Only "+p.stock+" in stock.",1)}
 if(c.qty<1)cart=cart.filter(x=>x.id!=id);render()}
const total=()=>cart.reduce((s,c)=>s+prod(c.id).price*c.qty,0);
function checkout(){if(!me){flash("Log in to check out.",1);return go("login")}
 if(me.role=="admin"){flash("Admins cannot place orders. Use a customer account.",1);return}go("checkout")}

/* ---------- Order + Payment ---------- */
function pay(){if(busy)return;const a=$("#ad").value.trim(),m=$("#pm").value,d=$("#pd").value.trim();
 if(a.length<8)return flash("Enter a full shipping address.",1);
 if(!d)return flash("Enter your "+(m=="UPI"?"UPI ID":"card number")+".",1);
 for(const c of cart)if(c.qty>prod(c.id).stock){flash("Stock changed for "+prod(c.id).name+". Update your
    cart.",1);return go("cart")}
 busy=true;$("#paybtn").textContent="Processing payment…";$("#paybtn").disabled=true;
 setTimeout(()=>{busy=false;
  if(d.endsWith("0000")){flash("Payment declined by the gateway. Your order was not placed. Try another payment
    method.",1);$("#paybtn").textContent="Pay now";$("#paybtn").disabled=false;return}
  const items=cart.map(c=>({id:c.id,name:prod(c.id).name,qty:c.qty,price:prod(c.id).price}));
  const o={id:db.nid++,userId:me.id,date:new Date().toLocaleString("en-IN"),items:items,total:total(),address:a,
    status:"Confirmed",payment:{id:"PAY"+db.nid,method:m,amount:total(),status:"Success"}};
  cart.forEach(c=>prod(c.id).stock-=c.qty);db.orders.unshift(o);save();cart=[];
  flash("Payment successful. Order #"+o.id+" is confirmed.");go("orders")},900)}
function cancel(id){const o=db.orders.find(x=>x.id==id);if(o.status!="Confirmed")return;
 o.status="Cancelled";o.items.forEach(i=>{const p=prod(i.id);if(p)p.stock+=i.qty});save();flash("Order #"+id+"
    cancelled. Stock restored.");render()}

/* ---------- Admin ---------- */
function addProd(){const n=$("#pn").value.trim(),c=$("#pc").value.trim()||"General",pr=+$("#pp").value,s=+$("#ps
    ").value;
 if(!n||pr<=0||s<0)return flash("Enter a name, a price above 0 and a stock of 0 or more.",1);
 db.products.push({id:db.nid++,name:n,cat:c,price:pr,stock:s,emoji:"*"});save();flash(n+" added.");render()}
function upd(id,f,v){const p=prod(id);v=+v;if(isNaN(v)||v<0)return flash("Enter a number of 0 or
    more.",1);p[f]=v;save();flash("Product updated.")}
function delProd(id){const p=prod(id);db.products=db.products.filter(x=>x.id!=id);cart=cart.filter(c=>c.id!=id);
    save();flash(p.name+" removed.");render()}
function setStatus(id,s){const o=db.orders.find(x=>x.id==id);if(s=="Cancelled"&&o.status!="Cancelled")o.items.fo
    rEach(i=>{const p=prod(i.id);if(p)p.stock+=i.qty});o.status=s;save();flash("Order #"+id+" is now
    "+s+".");render()}
function block(id){const u=db.users.find(x=>x.id==id);u.blocked=!u.blocked;save();flash(u.name+(u.blocked?"
    blocked.":" unblocked."));render()}

/* ---------- Views ---------- */
function nav(){const l=(v,t)=>'<button class="'+(view==v?"on":"")+'" onclick="go(\''+v+'\')">'+t+"</button>";
```

```
 let h='<span class="brand">ShopLite</span>';
 if(me&&me.role=="admin")h+=l("aprod","Products")+l("aord","Orders")+l("ausr","Users");
 else h+=l("shop","Shop")+l("cart","Cart ("+cart.reduce((s,c)=>s+c.qty,0)+")")+(me?l("orders","My orders"):"");
 h+=me?'<button onclick="logout()">Log out ('+esc(me.name.split(" ")[0])+")</button>":l("login","Log in");
 $("#nav").innerHTML=h}
function grid(){const L=db.products.filter(p=>(cat=="All"||p.cat==cat)&&p.name.toLowerCase().includes(q.toLowerC
    ase()));
 if(!L.length)return '<p class="mu">No products match your search. Clear the search or pick another
    category.</p>';
 return L.map(p=>'<div class="card"><div class="em">'+p.emoji+"</div><b>"+esc(p.name)+'</b><div
    class="mu">'+esc(p.cat)+" · "+(p.stock?p.stock+" in stock":"Out of stock")+'</div><div class="row"><span
    class="price">'+inr(p.price)+'</span><button class="btn" '+(p.stock?"":"disabled")+'
    onclick="add('+p.id+')">Add to cart</button></div></div>').join("")}
function setQ(v){q=v;$("#grid").innerHTML=grid()}
function setCat(v){cat=v;$("#grid").innerHTML=grid()}
const V={
 shop(){const cs=["All",...new Set(db.products.map(p=>p.cat))];
  return '<h2>Browse products</h2><div class="bar"><input placeholder="Search products" value="'+esc(q)+'"
    oninput="setQ(this.value)"><select onchange="setCat(this.value)">'+cs.map(c=>"<option"+(c==cat?"
    selected":"")+">"+esc(c)+"</option>").join("")+'</select></div><div class="grid"
    id="grid">'+grid()+"</div>"},
 login(){return '<h2>Log in or register</h2><div style="display:flex;gap:40px;flex-wrap:wrap"><div
    class="form"><b>I have an account</b><label>Email<input id="em" type="email"
    autocomplete="username"></label><label>Password<input id="pw" type="password"
    autocomplete="current-password"></label><button class="btn" onclick="login()">Log in</button></div><div
    class="form"><b>I am new here</b><label>Name<input id="rn"></label><label>Email<input id="re"
    type="email"></label><label>Password<input id="rp" type="password"></label><button class="btn ghost"
    onclick="register()">Create account</button></div></div>'},
 cart(){if(!cart.length)return '<h2>Your cart</h2><p class="mu">Your cart is empty. Browse the shop and add
    something you like.</p><button class="btn" onclick="go(\'shop\')">Browse products</button>';
  return '<h2>Your cart</h2><div class="wrap"><table><tr><th>Product</th><th>Price</th><th>Quantity</th><th>Subt
    otal</th></tr>'+cart.map(c=>{const p=prod(c.id);return"<tr><td>"+p.emoji+"
    "+esc(p.name)+"</td><td>"+inr(p.price)+'</td><td><button class="btn ghost"
    onclick="qty('+c.id+',-1)">−</button> '+c.qty+' <button class="btn ghost"
    onclick="qty('+c.id+',1)">+</button></td><td>'+inr(p.price*c.qty)+"</td></tr>"}).join("")+'</table></div><di
    v class="tot">Total '+inr(total())+'</div><div style="text-align:right"><button class="btn"
    onclick="checkout()">Check out</button></div>'},
 checkout(){return '<h2>Checkout</h2><div class="form"><label>Shipping address<textarea id="ad"
    rows="3">'+"</textarea></label><label>Payment method<select id=\"pm\"
    onchange=\"$('#pl').textContent=this.value=='UPI'?'UPI ID':'Card
    number'\"><option>Card</option><option>UPI</option></select></label><label><span id=\"pl\">Card
    number</span><input id=\"pd\" placeholder=\"Card number or UPI ID\"></label><div class=\"tot\"
    style=\"text-align:left\">Pay "+inr(total())+'</div><button class="btn" id="paybtn" onclick="pay()">Pay
    now</button></div>'},
 orders(){const L=db.orders.filter(o=>o.userId==me.id);if(!L.length)return '<h2>My orders</h2><p class="mu">You
    have not placed any orders yet.</p>';
  return"<h2>My orders</h2>"+L.map(o=>'<div class="card" style="margin-bottom:12px"><div class="row"
    style="margin:0"><b>Order #'+o.id+'</b><span class="tag">'+o.status+'</span></div><div
    class="mu">'+o.date+" · Paid by "+o.payment.method+" ("+o.payment.status+")</div><p>"+o.items.map(i=>esc(i.n
    ame)+" × "+i.qty).join(", ")+'</p><div class="mu">Ship to: '+esc(o.address)+'</div><div class="row"><span
    class="price">'+inr(o.total)+"</span>"+(o.status=="Confirmed"?'<button class="btn bad"
    onclick="cancel('+o.id+')">Cancel order</button>':"")+"</div></div>").join("")},
 aprod(){return '<h2>Manage products</h2><div class="bar"><input id="pn" placeholder="Name"><input id="pc"
    placeholder="Category"><input id="pp" type="number" placeholder="Price" style="max-width:110px;flex:0 1
    110px"><input id="ps" type="number" placeholder="Stock" style="max-width:110px;flex:0 1 110px"><button
    class="btn" onclick="addProd()">Add product</button></div><div
    class="wrap"><table><tr><th>Product</th><th>Price (₹)</th><th>Stock</th><th></th></tr>'+db.products.map(p=>"
    <tr><td>"+p.emoji+" "+esc(p.name)+'<div class="mu">'+esc(p.cat)+'</div></td><td><input type="number"
    value="'+p.price+'" onchange="upd('+p.id+',\'price\',this.value)"></td><td><input type="number"
    value="'+p.stock+'" onchange="upd('+p.id+',\'stock\',this.value)"></td><td><button class="btn bad"
    onclick="delProd('+p.id+')">Remove</button></td></tr>').join("")+"</table></div>"},
 aord(){if(!db.orders.length)return '<h2>Manage orders</h2><p class="mu">No orders have been placed yet.</p>';
  const S=["Confirmed","Packed","Shipped","Delivered","Cancelled"];
  return '<h2>Manage orders</h2><div class="wrap"><table><tr><th>Order</th><th>Customer</th><th>Items</th><th>To
    tal</th><th>Status</th></tr>'+db.orders.map(o=>{const
    u=db.users.find(x=>x.id==o.userId);return"<tr><td>#"+o.id+'<div
    class="mu">'+o.date+"</div></td><td>"+esc(u?u.name:"Deleted user")+"</td><td>"+o.items.map(i=>esc(i.name)+"
    × "+i.qty).join(", ")+"</td><td>"+inr(o.total)+'</td><td><select
    onchange="setStatus('+o.id+',this.value)">'+S.map(s=>"<option"+(s==o.status?"
    selected":"")+">"+s+"</option>").join("")+"</select></td></tr>"}).join("")+"</table></div>"},
 ausr(){return '<h2>Manage users</h2><div class="wrap"><table><tr><th>Name</th><th>Email</th><th>Role</th><th></
    th></tr>'+db.users.map(u=>"<tr><td>"+esc(u.name)+"</td><td>"+esc(u.email)+'</td><td><span
    class="tag">'+u.role+"</span></td><td>"+(u.role=="admin"?"":'<button class="btn
    '+(u.blocked?"ghost":"bad")+'" onclick="block('+u.id+')">'+(u.blocked?"Unblock":"Block")+"</button>")+"</td>
    </tr>").join("")+"</table></div>"}
};
function render(){if(!me&&["orders","aprod","aord","ausr"].includes(view))view="login";
 if(me&&me.role=="admin"&&["shop","cart","checkout","orders"].includes(view))view="aprod";
 if(me&&me.role!="admin"&&["aprod","aord","ausr"].includes(view))view="shop";
 $("#app").innerHTML=V[view]();nav();
}
render();
```

9. CONCLUSION

The Online Shopping System is a comprehensive web application that successfully replaces the manual, shop-visit based buying process with a digital one. Built with HTML, CSS and JavaScript, the platform provides role-based access for customers and administrators, enabling seamless product browsing, cart management, checkout, payment, order tracking and administration. The application features intuitive screens, instant validation messages and admin oversight tools, creating an integrated environment where customers can shop without visiting a store and administrators can control products, orders and users.

The system was analysed and designed using use case, class, activity, sequence, component and deployment diagrams in IBM Rational Software Architect Designer, and every diagram was carried into the implementation: the class diagram became the data model, the use cases became screens and functions, and the activity and sequence diagrams became the checkout and payment workflow. Validation ensures that stock is never oversold, that a declined payment never creates an order and that cancelling an order restores stock. All 31 test cases passed.

The Online Shopping System demonstrates proficiency in front-end development and UML-based design and serves as a valuable platform for small shops, college stores and startups, showcasing modern web development practices and user-centred design principles.

9.1 Future Scope

- Replace the browser storage with a Node.js or Java server and a MySQL database, hosted over HTTPS, as shown in the deployment diagram.
- Store passwords as salted hashes and use token-based sessions.
- Integrate a real payment gateway (UPI, cards, net banking) with receipts by e-mail or SMS.
- Add product images, reviews and ratings, wishlists and discount coupons.
- Add order tracking with a delivery partner and returns and refunds.
- Develop a mobile application and add reports and analytics for sales and stock.

10. REFERENCES

Documentation & Official Sources

- MDN Web Docs (HTML, CSS, JavaScript, Web Storage API) – https://developer.mozilla.org/
- Unified Modeling Language Specification, Object Management Group – https://www.omg.org/spec/UML/
- IBM Rational Software Architect Designer – https://www.ibm.com/products/rational-software-architect-designer
- Visual Studio Code and Live Server extension – https://code.visualstudio.com/
- OWASP Top 10 Web Application Security Risks – https://owasp.org/www-project-top-ten/

Books

- Grady Booch, James Rumbaugh, Ivar Jacobson, *The Unified Modeling Language User Guide*, Addison-Wesley.
- Martin Fowler, *UML Distilled: A Brief Guide to the Standard Object Modeling Language*, Addison-Wesley.